

**SQLI
DIGITAL
EXPERIENCE**

SCRUM DEV FORMATION

J1

V 3.1.0

COLLABORATION



Respect des horaires



Pas de téléphone
ou d'ordinateur



Présence



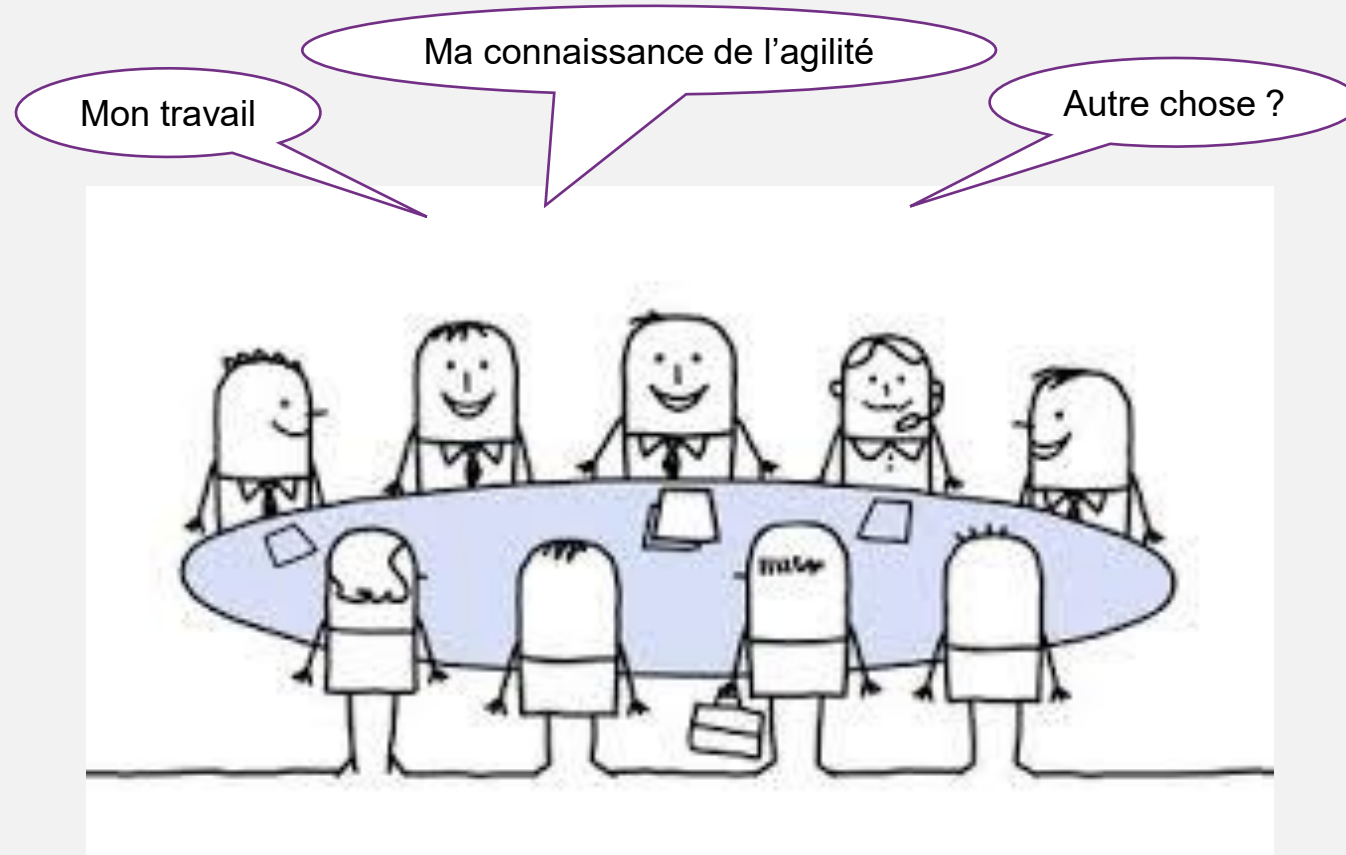
Photo



Participation



PARTICIPANTS





LÉGENDE

PSD1

Information à maîtriser dans le cadre de la certification

EXO

Exercice

IMP

Information complémentaire à l'implémentation
et la vie des projets agiles



ATTENTES DE LA FORMATION

EXERCICE

- DÉFINISSEZ VOS OBJECTIFS DE FORMATION SOUS LA FORME D'UNE « USER STORY »
- ÉCRIVEZ-LES SUR UN POST-IT
- METTEZ-LES SUR LE TABLEAU

EN TANT QUE
<TYPE D'UTILISATEUR>

JE VEUX
<UN OBJECTIF>

POUR QUE
<LA VALEUR MÉTIER>



POURQUOI VOUS ÊTES LÀ :

POUR VOUS PRÉPARER À PASSER LA CERTIFICATION PSD1

➤ PLAN DE LA FORMATION

- Constats et historique
- Le manifeste agile
- Comprendre et appliquer le cadre de Scrum
- L'équipe Scrum :
 - Le Product Owner
 - Le Scrum Master
 - Les Développeurs
- Les rituels
- Les artefacts
- Développement des personnes et des équipes
- Les autres méthodes agiles
- Développer et livrer des produits de manière professionnelle
 - Développement logiciel itératif
 - La qualité et Scrum
 - Pair-programming
 - Clean Code
 - La dette technique
 - Qualimétrie
 - Source control & CI/CD
- Training
 - One Scrum
 - Open Assesment

CONSTATS ET HISTORIQUE





DÉVELOPPEMENT TRADITIONNEL DE LOGICIELS

QUELQUES PROBLÈMES COURAMMENT RENCONTRÉS ...

➤ SPÉCIFICATIONS PAS CLAIRES

- Peur d'aller à la phase d'après

➤ SPÉCIFICATIONS CHANGENT

- Le changement est de plus en plus couteux
- Les clients n'ont pas ce qu'ils souhaitent

➤ PROJET QUI PREND TROP DE TEMPS

- 32% des projets sont livrés avec succès dans les délais
- Les longues durées diffèrent les revenus (Source : Standish Report 2009)

➤ PAS DE TEMPS POUR TESTER

- Assurance qualité reléguée au second plan
- Les intégrations tardives impliquent des échecs tardifs

➤ TEMPS DÉPENSÉ SUR DES FONCTIONNALITÉS INUTILES

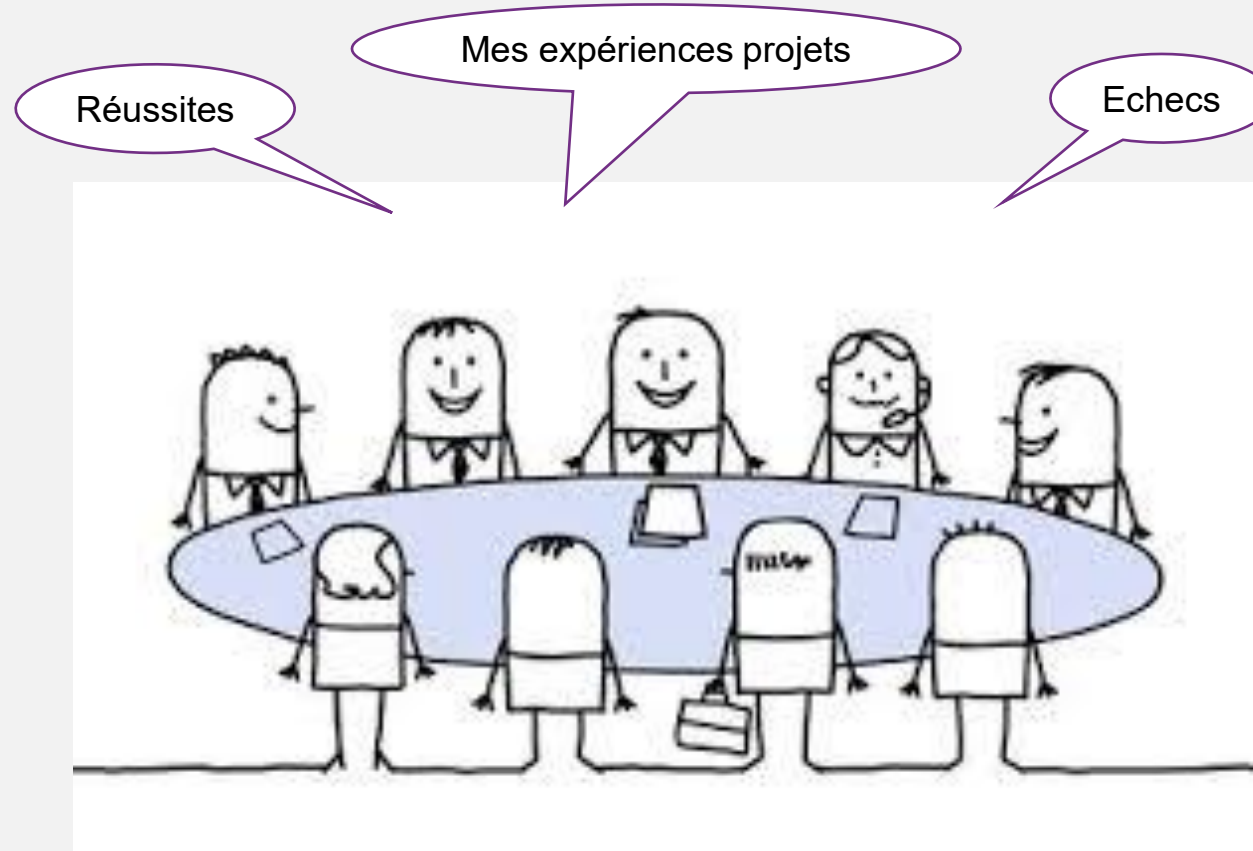
- 52% des spécifications sont implémentées
- 64% des fonctionnalités sont rarement utilisées (Source : Standish Report 2003)

➤ FAIBLE VISIBILITÉ SUR L'AVANCEMENT

- Pourcentage de tâches réalisées non suffisant
- 44 % des projets ont abouti, hors délai, hors budget, ou sans respecter les exigences initiales (Source : Standish Report 2003)



RETOUR D'EXPÉRIENCE PROJET





CAUSES PRINCIPALES DES ÉCHECS

CONSTATS

➤ BEAUCOUP DE CHANGEMENTS DUS

- ... au client / utilisateur qui ne sait pas tout au début
- ... à un feedback tardif
- ... une perte d'informations entre les interfaces organisationnelles

➤ ENVIRONNEMENT VUCA DÛ À

- ... un marché global que l'on ne peut pas contrôler
- ... une numérisation menant à un flux d'information rapide
- ... un comportement des utilisateurs finaux imprédictible

➤ L'INCERTITUDE ET LA COMPLEXITÉ SONT GÉNÉRALEMENT IMPORTANTES

➤ IL EST IMPOSSIBLE DE TOUT COMPENSER PAR UNE IMPORTANTE PLANIFICATION AU LANCEMENT DU PROJET



CRÉER LE BON PRODUIT

... Franchement, les approches agiles effraient les bureaucrates d'entreprise - du moins ceux qui se contentent d'appliquer des processus pour le plaisir de le faire au lieu d'**essayer de faire au mieux pour le "client" et de livrer quelque chose de rapide, de tangible et "comme promis"** - parce qu'ils ne savent plus où se cacher.

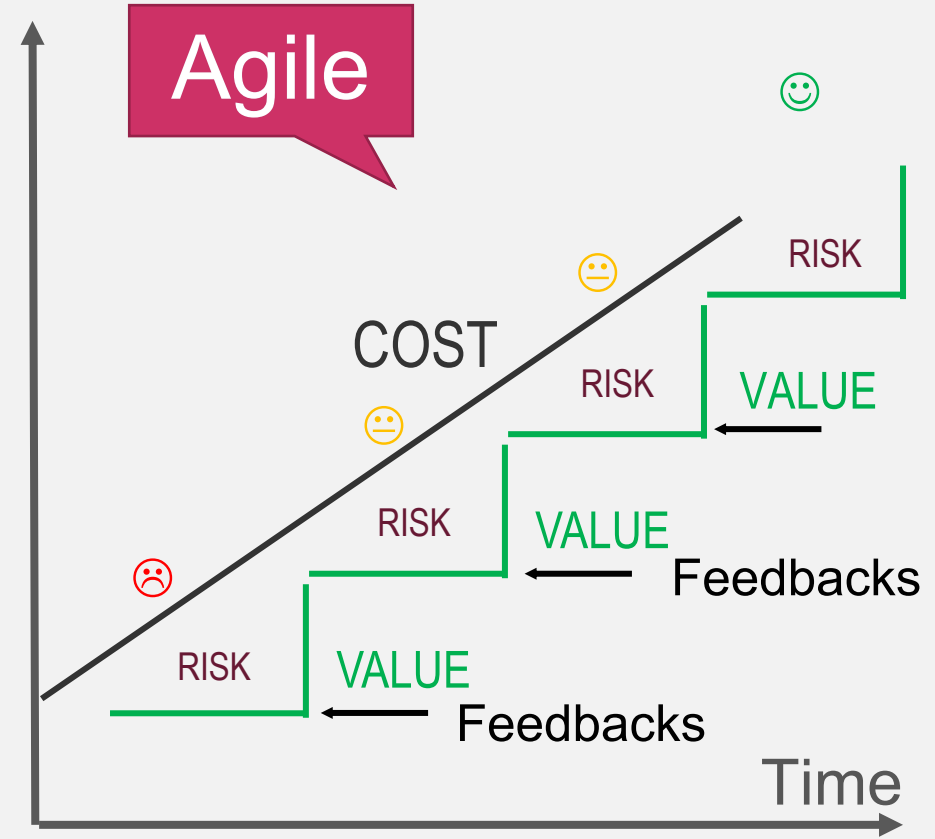
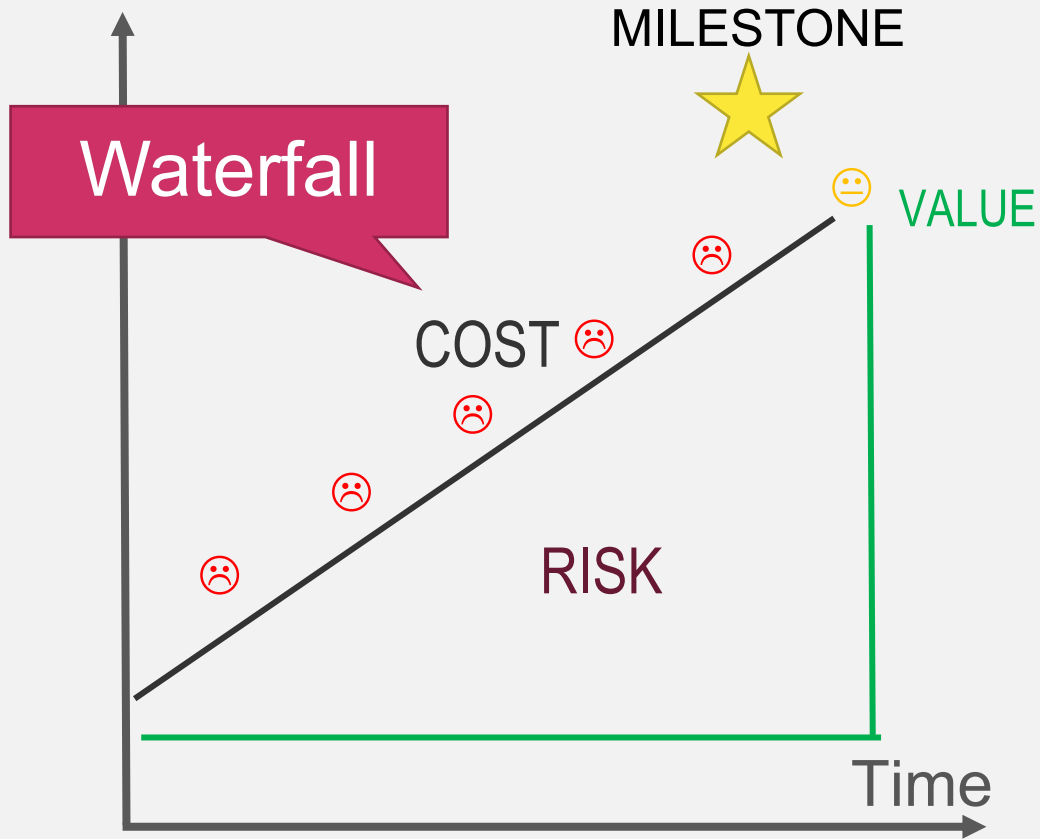
Jim Highsmith, for the Agile Alliance

<https://agilemanifesto.org/history.html>



COÛT, VALEUR ET RISQUE

... SONT SUR UN BATEAU



Fournir progressivement de la valeur au client

Plus les cycles sont courts, plus l'apprentissage est rapide.

LE MANIFESTE AGILE



AGILE HISTORY

- 1986 : ITERATIVE AND INCREMENTAL DEVELOPMENT MODEL FROM BARRY W. BOEHM
- 1991 : RAPID APPLICATION DEVELOPMENT (RAD) FROM JAMES MARTIN
- 1995 : AGILE METHODS
- 2001 : AGILE MANIFESTO

RAD

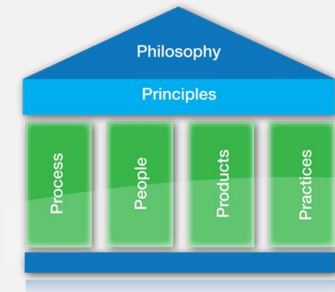
Rapid Application Development



ASD

Adaptive Software Development

The DSDM Agile Project Framework

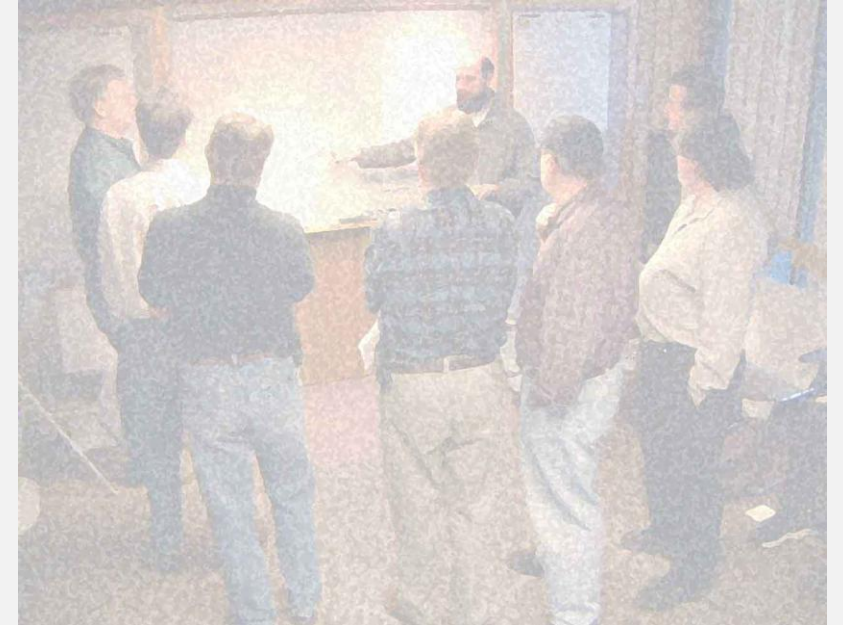


Crystal Clear



CONTEXTE DU MANIFESTE AGILE

- Quand : du 11 au 13 février 2001
- Où : Montagne Wasatch (Utah – états unis)
- Qui : 17 experts
- Résultat : promouvoir une approche différente du développement logiciel en s'attachant à délivrer de la meilleure façon possible ce qui a de la valeur pour le Client





L'AGILITÉ EST BASÉE SUR 4 VALEURS

NOUS DÉCOUVRONS COMMENT MIEUX DÉVELOPPER DES LOGICIELS PAR LA PRATIQUE ET EN AIDANT LES AUTRES À LE FAIRE.

CES EXPÉRIENCES NOUS ONT AMENÉS À VALORISER :

Les individus et leurs interactions plus que les processus et les outils

Des logiciels opérationnels plus qu'une documentation exhaustive

La collaboration avec les clients plus que la négociation contractuelle

L'adaptation au changement plus que le suivi d'un plan

Nous reconnaissons la valeur des seconds éléments,
mais privilégions les premiers.



... ET 12 PRINCIPES

1 Satisfy the **customer**



2 Welcome **change**



3 Deliver **frequently**

Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
story	story	story	story	story
story	story	story	story	story
story	story	story	story	story

4 Work **together**



5 Trust and **support**



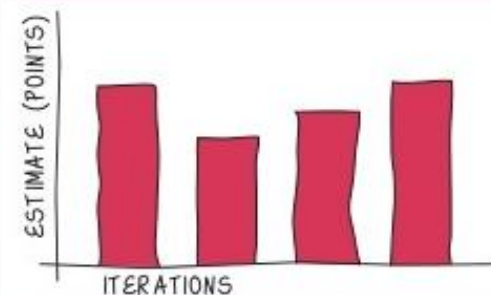
6 Face-to-face **conversation**



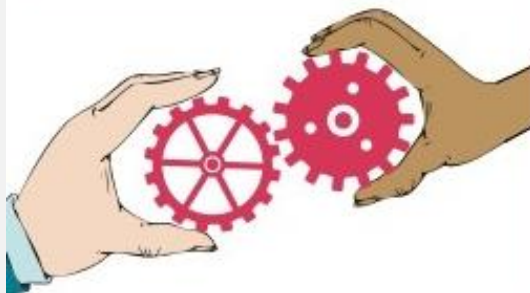
7 Working **software**



8 Sustainable **development**



9 Continuous **attention**



10 Maintain **simplicity**



11 Self-organizing **teams**



12 Reflect and **adjust**



PANORAMA MÉTHODOLOGIES AGILE

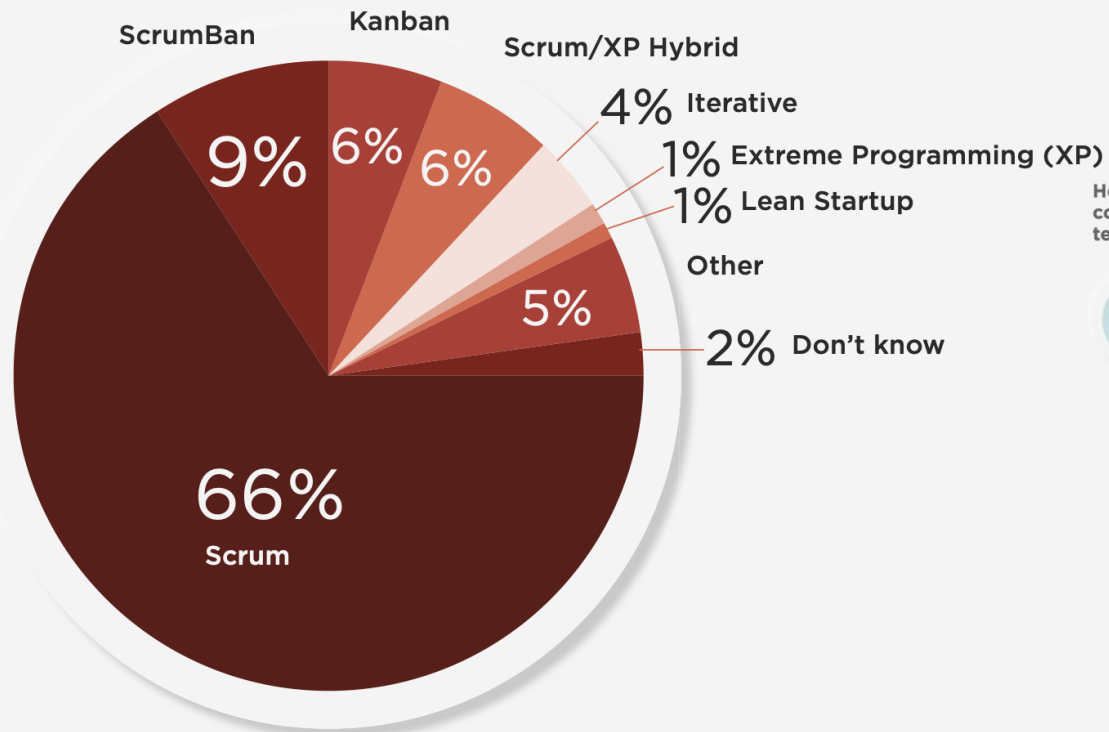




THE 15TH STATE OF AGILE SURVEY

CONDUCTED BETWEEN FEBRUARY AND APRIL 2021

Which agile methodology do you follow most closely at the team level?



15th State of Agile Report

Between February and April 2021.

Individuals across a broad range of industries in the global software development communities were invited to participate.

A total of **4,182** responses were received with **1,382** complete survey responses collected

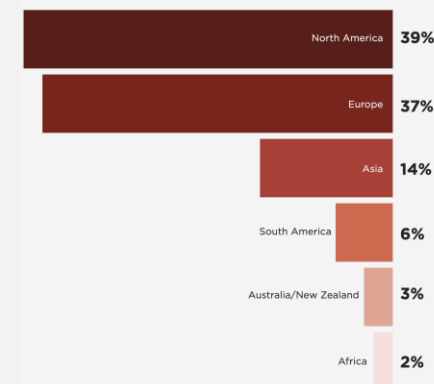
How many people in your organization (including contractors) are responsible for planning, developing, testing, and delivering software?



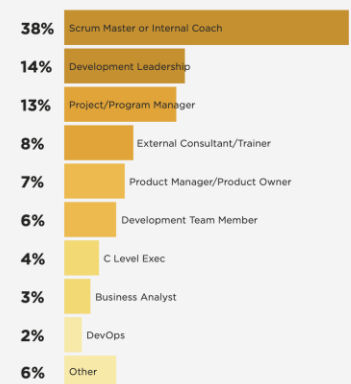
How many employees are in your entire company?



Where are you located?



Which role best describes your current position?

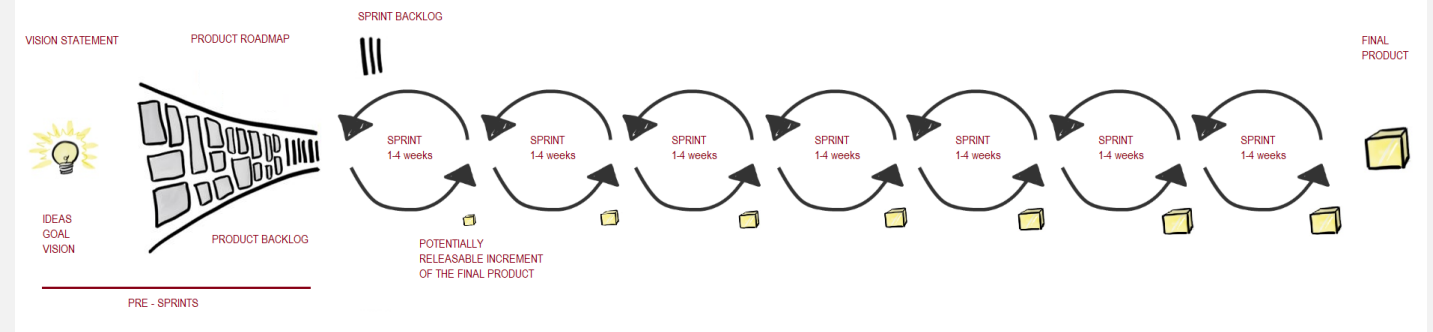


COMPRENDRE ET APPLIQUER LE CADRE DE SCRUM





TIMELINE



> SCRUM :

- Scrum est un Framework (cadre de travail) léger qui aide les personnes, les équipes et les organisations à générer de la valeur grâce à des solutions adaptatives pour des problèmes complexes.

> SCRUM A BESOIN D'UN SCRUM MASTER POUR FAVORISER UN ENVIRONNEMENT OÙ :

- Un Product Owner priorise le travail pour un problème complexe dans le Product Backlog.
- La Scrum Team transforme une sélection de travail en un incrément de valeur lors d'un Sprint.
- La Scrum Team et ses parties prenantes inspectent les résultats et s'adaptent pour le prochain Sprint.
- Répéter

> SCRUM EST :

- Léger
- Volontairement incomplet
- Repose sur l'intelligence collective



RUN SCRUM

EXERCICE

➤ BUT:

- Réaliser le maximum d'avions en papier en 3 minutes

➤ CRITÈRES D'ACCEPTANCE:

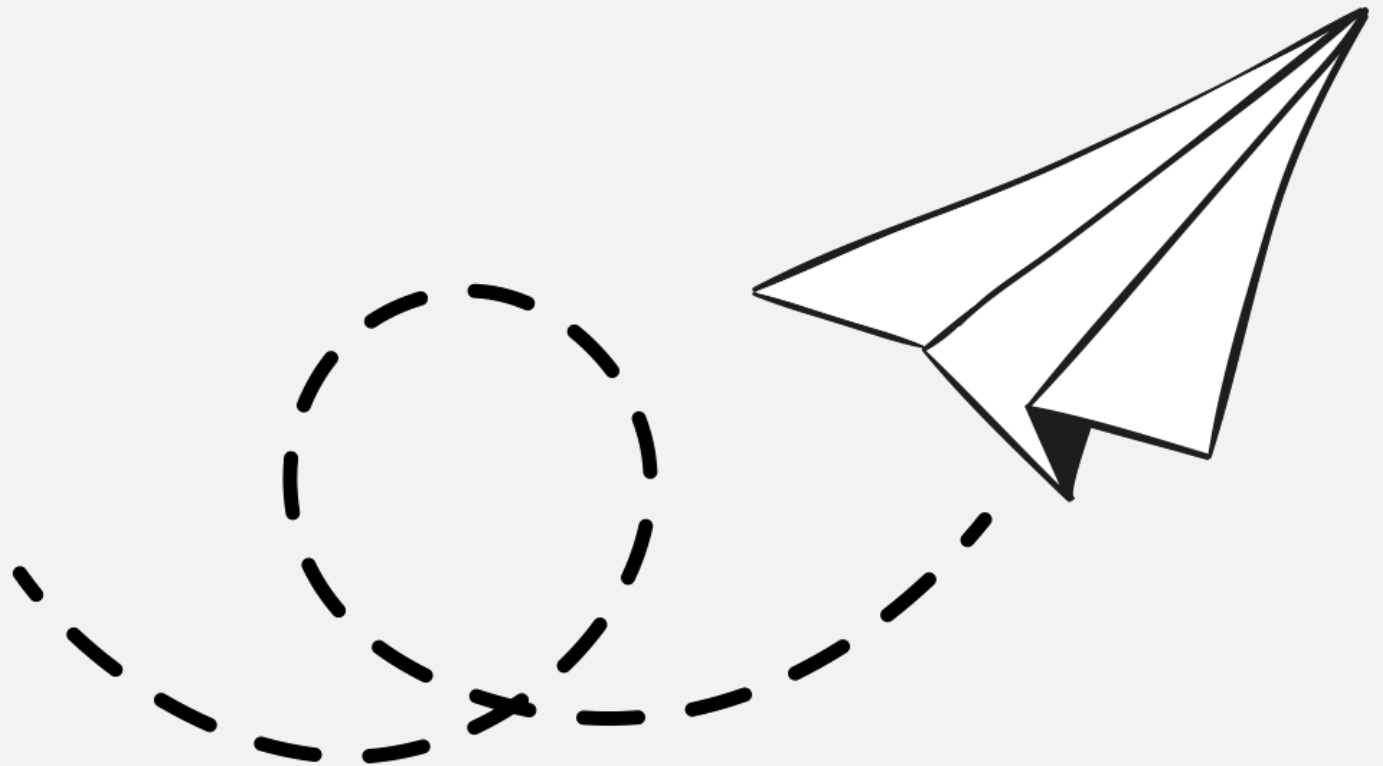
- Format A5
- Vol au moins sur 3 mètres

➤ CONTRAINTES:

- Un pli par personne
- Le Product Owner teste

➤ TEMPS:

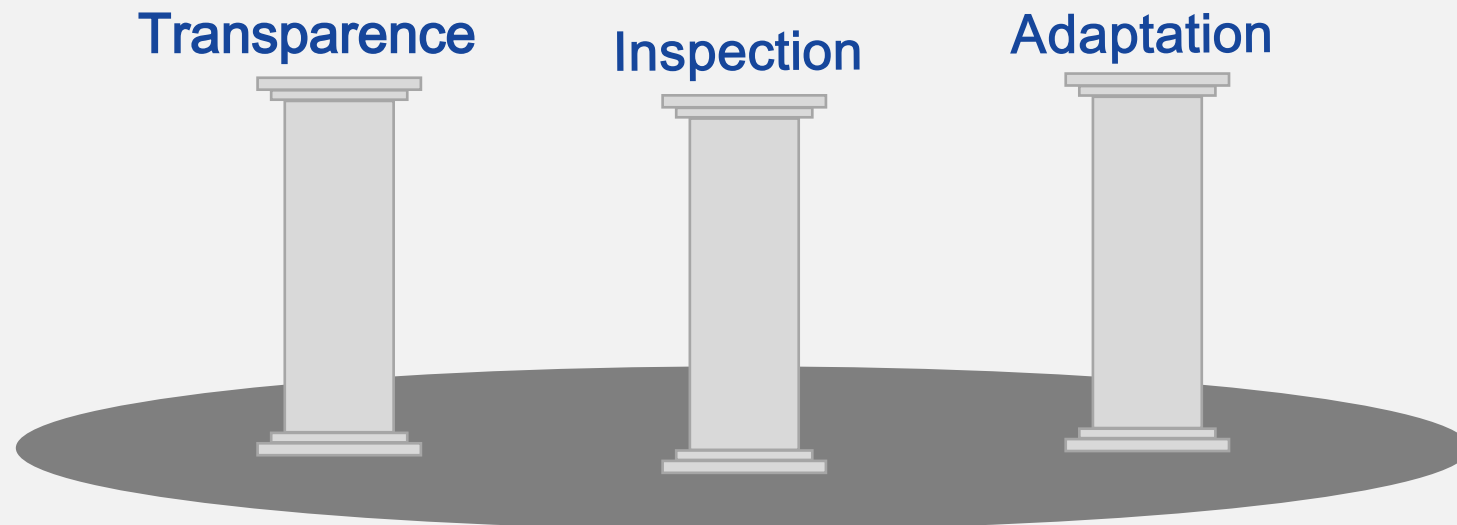
- 3 sprints
 - Préparation: 1 minute
 - Exécution: 3 minutes
 - Debrief: 2 minutes





LES PILIERS DE SCRUM

- SCRUM EST FONDÉ SUR L'EMPIRISME ET LE LEAN THINKING.
- L'EMPIRISME AFFIRME QUE LA CONNAISSANCE VIENT DE L'EXPÉRIENCE ET DE LA PRISE DE DÉCISIONS EST BASÉE SUR DES FAITS OBSERVÉS.
- LE LEAN THINKING RÉDUIT LE GASPILLAGE ET SE FOCALISE SUR L'ESSENTIEL.





LES 5 VALEURS DE SCRUM

L'UTILISATION RÉUSSIE DE SCRUM DÉPEND DE LA CAPACITÉ DES PERSONNES À MIEUX VIVRE CES CINQ VALEURS

> COURAGE

- les membres de L'équipe Scrum ont le courage de faire ce qu'il faut, de travailler sur des problèmes difficiles.

> FOCUS

- leur objectif principal est le travail du Sprint pour progresser le plus possible vers ces objectifs

> COMMITMENT (=ENGAGEMENT)

- L'équipe Scrum s'engage à atteindre ses objectifs et à se soutenir mutuellement

> RESPECT

- Les membres de L'équipe Scrum se respectent mutuellement pour être des personnes compétentes et indépendantes, et sont respectés en tant que tels par les personnes avec lesquelles ils travaillent.

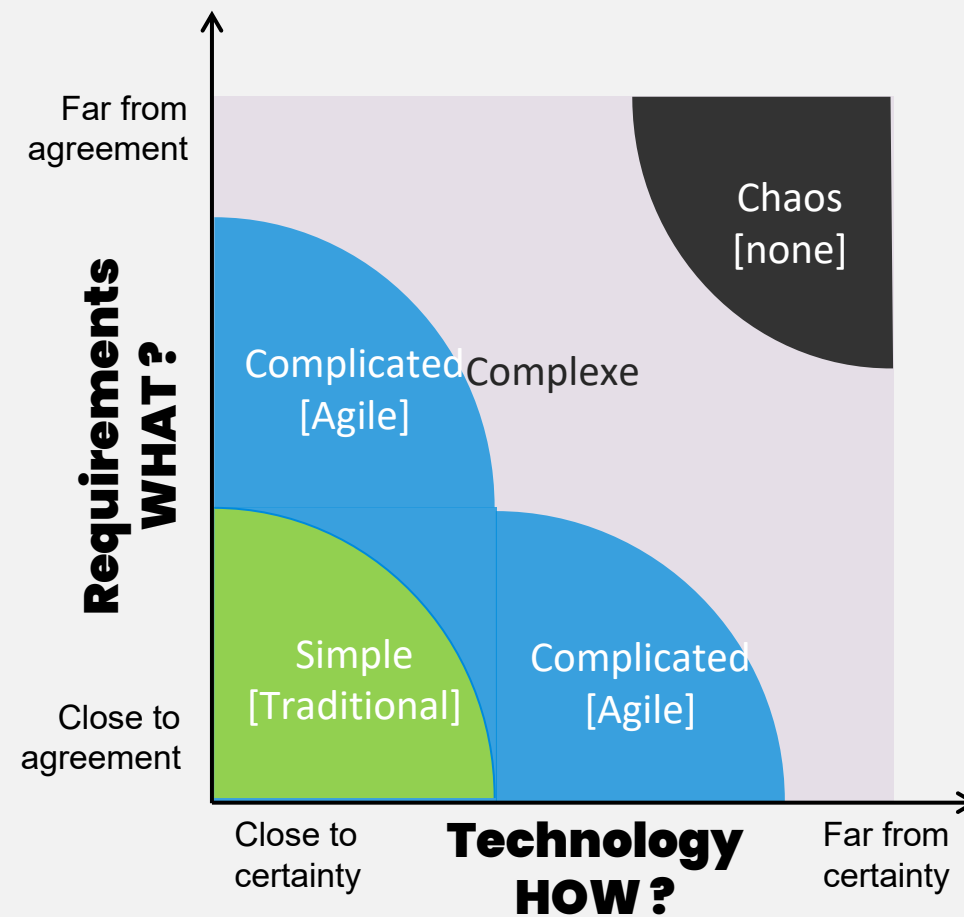
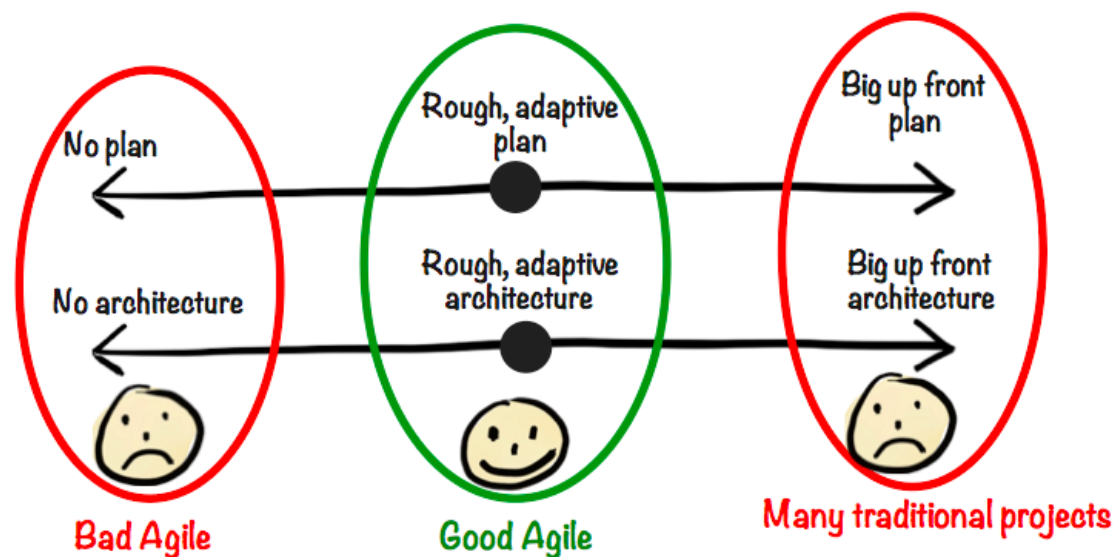
> OPENNESS (=OUVERTURE)

- L'équipe Scrum et ses parties prenantes sont ouvertes sur le travail et les défis à relever.



EMPIRISME

Don't go overboard with Agile!

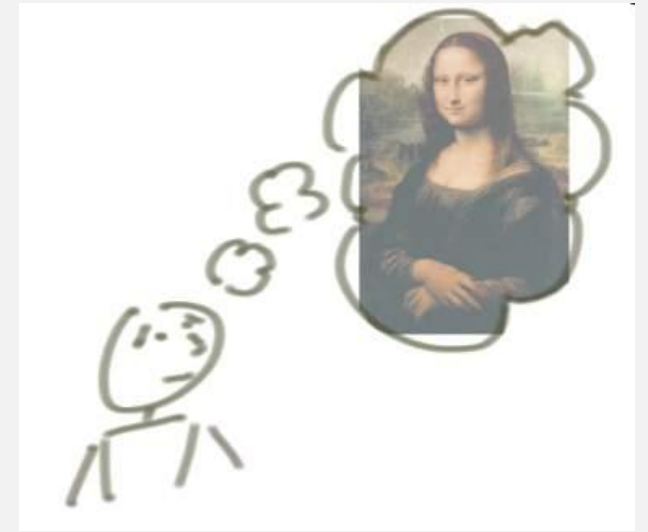


Matrice de Ralph STACEY



INCRÉMENTAL

- « INCRÉMENTER » EXIGE D'AVOIR UNE IDÉE GLOBALE DU PRODUIT.
- ET LE FAIRE À TEMPS NÉCESSITE UNE PLANIFICATION PRÉCISE.
- « INCRÉMENTER » PERMET DE CONSTRUIRE MORCEAU PAR MORCEAU



1



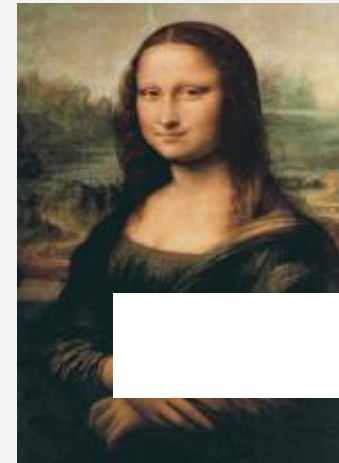
2



3



4



5





ITÉRATIF

- UNE VERSION PLUS ITÉRATIVE VOUS PERMET DE PASSER D'UNE IDÉE VAGUE À UNE RÉALISATION EN APPORTANT DES CORRECTIONS DE PARCOURS AU FUR ET À MESURE.
- « ITÉRER » PERMET DE CONSTRUIRE UNE VERSION BRUTE, DE LA VALIDER, PUIS DE CONSTRUIRE PROGRESSIVEMENT LA QUALITÉ.



1



2



3



4



5



ADAPTATIF

- SCRUM EST À LA FOIS ITÉRATIF ET INCRÉMENTAL, IL EST ADAPTATIF.

Agile = Iterative + Incremental

Don't try to get it all right from the beginning

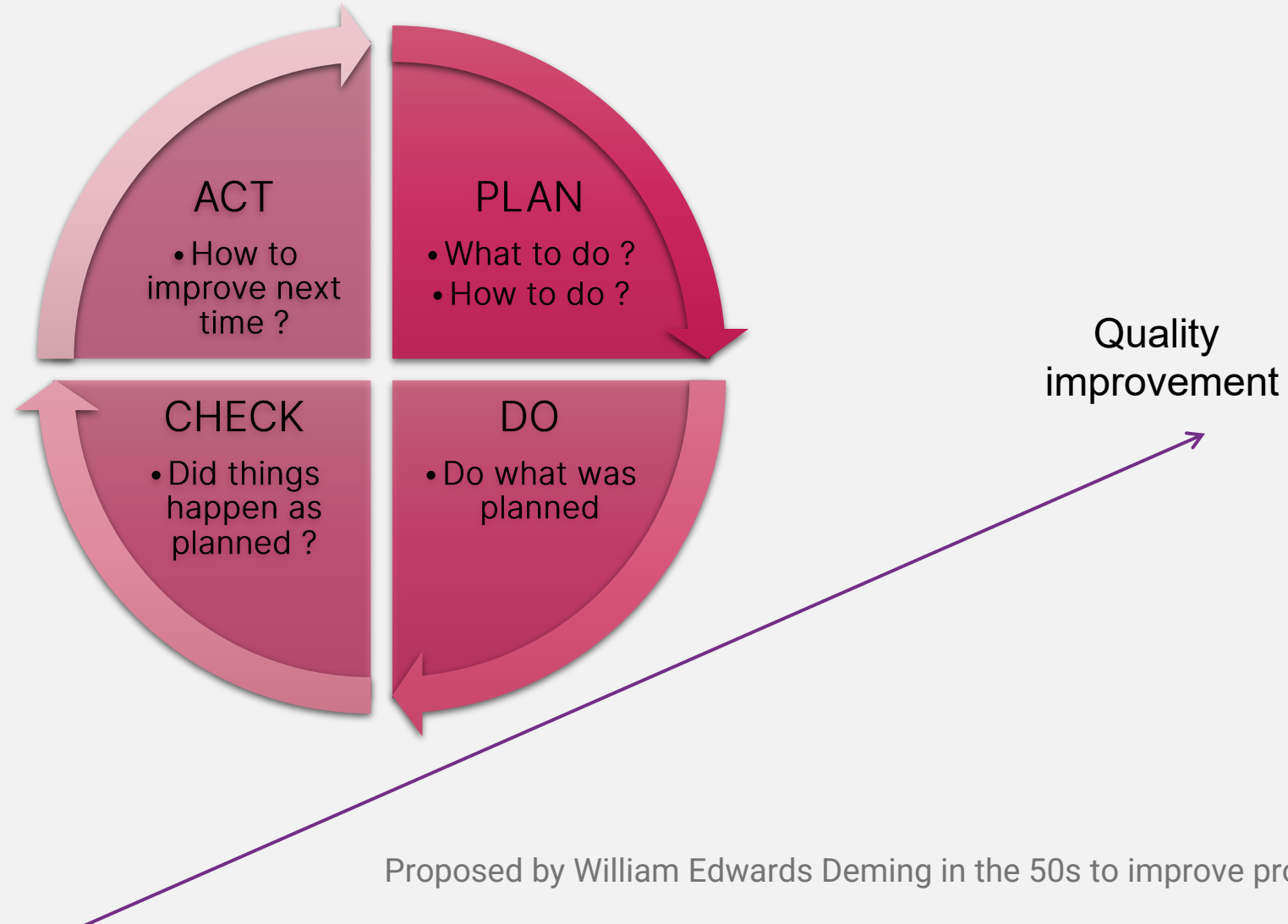
Don't build it all at once



cumulative
business value



AMÉLIORATION CONTINUE



Proposed by William Edwards Deming in the 50s to improve project quality

L'ÉQUIPE SCRUM





LA SCRUM TEAM : 3 ROLES

UN GROUPE UNI DE PROFESSIONNELS SE CONCENTRANT SUR UN SEUL OBJECTIF À LA FOIS, L'OBJECTIF DU PRODUIT.



1 personne à temps plein ou à temps partiel, orientée vers le business (métier et marché)

La voix unique du client qui fixe la vision, hiérarchise le travail et définit les critères de réussite.

Plusieurs personnes à temps plein

Créer de la valeur pour le client.

1 personne à temps plein ou à temps partiel. Coach et facilitateur Scrum

Le leader de la situation qui facilite le processus et supprime les obstacles.



LA SCRUM TEAM : 3 RÔLES

2 CARACTÉRISTIQUES

➤ PLURIDISCIPLINAIRES

- Les membres ont toutes les compétences nécessaires pour créer de la valeur à chaque Sprint.
- Des interventions ponctuelles d'experts externes sont possibles, mais cela doit toujours être fait dans un objectif d'acquisition de connaissances par l'équipe.

➤ AUTOGÉRÉS

- Ils décident en interne qui fait quoi, quand et comment.



TAILLE DE L'ÉQUIPE

➤ L'ÉQUIPE SCRUM DOIT ÊTRE

- Suffisamment petite pour rester réactive
- Assez grande pour accomplir un travail significatif durant le Sprint

=> Habituellement pas plus de **dix personnes**.

➤ CONSTAT : LES PETITES ÉQUIPES COMMUNIQUENT MIEUX ET SONT PLUS PRODUCTIVES

➤ ET SI LES SCRUM TEAMS DEVIENNENT TROP GRANDES ?

- Elles devraient envisager de se réorganiser en plusieurs Scrum Teams cohérentes, chacune axée sur le même produit.
- Par conséquent, ils doivent partager le même Objectif de Produit, le même Product Backlog et le même Product Owner.



RESPONSABILITÉ DE LA SCRUM TEAM

- **L'ÉQUIPE SCRUM EST RESPONSABLE DE TOUTES LES ACTIVITÉS LIÉES AUX PRODUITS :**
 - Collaboration des parties prenantes
 - Vérification, maintenance
 - Exploitation
 - Expérimentation
 - Recherche et développement
 - et tout ce qui pourrait être nécessaire...
- **ILS SONT STRUCTURÉS ET HABILITÉS PAR L'ORGANISATION À GÉRER LEUR PROPRE TRAVAIL**
- **TRAVAILLER DANS LES SPRINTS À UN RYTHME SOUTENABLE AMÉLIORE LA CONCENTRATION ET LA COHÉRENCE DE L'ÉQUIPE SCRUM.**
- **TOUTE L'ÉQUIPE SCRUM EST RESPONSABLE DE LA CRÉATION D'UN INCRÉMENT DE VALEUR ET UTILE DURANT CHAQUE SPRINT**



GESTION DE PROJET

Qui permet la réussite d'un projet SCRUM ?
Indiquez qui est responsable (R), et qui y contribue (C)

Nr	Tâche ou responsabilité	PO	SM	Dev. Team	Autre	Inutile
1	Définir et garantir l'objectif du projet/produit (sa vision) et communiquer auprès de l'équipe et des acteurs du projet.	R				
2	Susciter l'intérêt / investissement des clients, acteurs business ... pour le projet / produit.	R				
3	Définition du plan projet global initial : délais, milestones, livrables, scope du projet, le budget, les intervenants, ...	R		C		
4	Définition du plan projet global en détail : délais, milestones, livrables, scope du projet, le budget, les intervenants, ...					R
5	Mise en place des outils et suivi de projet (coût, scope, délai, qualité).	R				
6	Identifier et évaluer les risques du projet, prise de décision sur le plan d'atténuation des risques.	R	C	C	C	
7	Obtenir les accords et planifier les ressources projet et le nombre de jours/homme.	R	C		C	
8	Créer un environnement de travail optimal pour que l'équipe collabore et délivre.		R			
9	Réaliser, délivrer les documents et le reporting liés à la gestion de projet et à la gouvernance;		C	R	C	
10	Développer et maintenir la documentation projet/produit.			R		
11	Planifier et gérer les activités quotidiennes de l'équipe.			R		
12	Suivre et communiquer le statut projet / produit (suivi par rapport au plan initial).	R				
13	Gérer la communication vis-à-vis des sponsors, acteurs projet, clients, comité de pilotage.	R				



GESTION DE PROJET

Qui permet la réussite d'un projet SCRUM ?
Indiquez qui est responsable (R), et qui y contribue (C)

Nr	Tâche ou responsabilité	PO	SM	Dev. Team	Autre	Inutile
14	Gérer les problèmes opérationnels qui font obstacle à la délivrance du projet pour l'équipe.	C	R	C	C	
15	Organiser et faciliter les réunions Scrum.		R			
16	Mener la réunion quotidienne Daily Scrum.			R		
17	S'occuper de la gestion d'équipe (motivation, coaching, formation, offre salariale, primes de reconnaissance, ressources humaines ...)		C	C	R	
18	Accepter et Approuver les livrables projet / produit.	R				
19	Mettre à jour le Sprint Burndown Chart, et le tableau des tâches.			R		
20	Faire la démonstration des résultats du Sprint auprès des Stakeholders, et le Product Owner lors de la revue du Sprint.			R		
21	Gérer les dépendances avec les équipes externes et les fournisseurs.	R	R	R		
22	Prioriser le product backlog en fonction de la valeur ajoutée business, risques, les coûts ...	R				
23	Estimer la charge de travail de l'équipe.			R		
24	S'assurer de l'implication Scrum dans l'organisation, et former les équipes impactées à Scrum.		R			
25	Prendre les décisions sur le scope à délivrer et dans chaque Sprint.			R		
26	S'assurer des améliorations continues sur tous les aspects du projet.	C	R	C	C	

PRODUCT OWNER





PRODUCT OWNER

Fiction	Réalité
Le Product Owner est le chef de projet.	Le Product Owner crée et gère uniquement le Product Backlog, mais ne gère pas les activités quotidiennes de l'équipe.
Le Product Owner est un représentant du client.	Le Product Owner est l'une des personnes de l'organisation en charge de la réalisation du produit final, et le point de contact avec le client.



Product Owner



PRODUCT OWNER

- **RESPONSABLE DE MAXIMISER LA VALEUR DU PRODUIT RÉSULTANT DU TRAVAIL DE LA SCRUM TEAM.**
 - La manière de procéder peut varier considérablement selon les organisations, les Scrum Teams et les individus.
- **LE PRODUCT OWNER EST ÉGALEMENT RESPONSABLE DE LA GESTION EFFICACE DU PRODUCT BACKLOG, QUI COMPREND :**
 - Développer et communiquer explicitement l'Objectif de Produit
 - Créer et communiquer clairement les éléments du Product Backlog
 - Ordonner les éléments dans le Product Backlog ; et,
 - S'assurer que le Product Backlog est transparent, visible et compréhensible.



PRODUCT OWNER

- **LE PRODUCT OWNER PEUT EFFECTUER LE TRAVAIL CI-DESSUS OU PEUT DÉLÉGUER LA RESPONSABILITÉ À D'AUTRES.**
 - De toute façon, le Product Owner en demeure responsable.
- **POUR QUE LES PRODUCT OWNERS RÉUSSISSENT, TOUTE L'ORGANISATION DOIT RESPECTER LEURS DÉCISIONS.**
 - Ces décisions sont visibles dans le contenu et l'ordre du Product Backlog, et via un incrément vérifiable lors de la Sprint Review.
- **LE PRODUCT OWNER EST UNE PERSONNE, ET NON UN COMITÉ.**
- **LE PRODUCT OWNER PEUT REPRÉSENTER LES BESOINS DE NOMBREUSES PARTIES PRENANTES DANS LE PRODUCT BACKLOG.**
 - Ceux qui souhaitent modifier le Product Backlog peuvent le faire en essayant de convaincre le Product Owner.



PRODUCT OWNER COMPÉTENCES

Expert

Produit et entreprise
Pas nécessairement technique

Décideur

Il est légitime et disponible
Il a le pouvoir de décision

Communiquant

Expliquer le besoin à l'équipe de développement
Capter les besoins réels du client et/ou des utilisateurs
Communiquer sur l'avancement du produit
Défendre le travail de l'équipe auprès des managers

Ouvert

Comprendre les activités de l'équipe de développement
Sensible aux bonnes pratiques et à la qualité
Sensible à l'expérience de l'utilisateur

SCRUM MASTER





SCRUM MASTER

Fiction	Réalité
Le Scrum Master est comme un chef de projet.	Le Scrum Master s'assure que le framework Scrum est mis en œuvre efficacement. Il n'y a pas d'équivalent au Chef de Projet traditionnel dans un projet Scrum.



LE SCRUM MASTER EST...

- **CHARGÉ DE METTRE EN PLACE SCRUM TEL QUE DÉFINI DANS LE GUIDE SCRUM.**
 - Les Scrum Masters remplissent leur rôle en aidant tout le monde à comprendre la théorie et la pratique Scrum, à la fois au sein de L'équipe Scrum et de l'organisation.
- **RESPONSABLE DE L'EFFICACITÉ DE L'ÉQUIPE SCRUM.**
 - Les Scrum Masters remplissent leur rôle en permettant à L'équipe Scrum d'améliorer ses pratiques liées à Scrum.
- **UN VÉRITABLE LEADER QUI SERT L'ÉQUIPE SCRUM ET L'ENSEMBLE DE L'ORGANISATION.**
- **LE SCRUM MASTER SERT DE PLUSIEURS FAÇONS :**
 - La Dev Team
 - Le Product Owner
 - L'organisation



SCRUM MASTER COMPÉTENCES

- CONNAÎTRE SCRUM
- AVOIR UN TALENT DE COMMUNICATION
- COMPRENDRE LA TECHNIQUE
- SOUTENIR LES ÉQUIPES DANS LEURS DIFFICULTÉS
- RÉSOUDRE LES CONFLITS
- ANTICIPER LES RISQUES
- IDENTIFIER LES PROBLÈMES
- AVOIR UNE POSITION DE *SERVENT LEADER* POUR RÉSOUDRE LES PROBLÈMES ET AMÉLIORER LES PROCESSUS



DÉVELOPPEURS





LES DÉVELOPPEURS

Fiction	Réalité
Les développeurs sont libres de faire ce qu'ils veulent.	Les développeurs travaillent dans un cadre productif et prédéfini et le Scrum Master s'assure qu'ils suivent Scrum.
Scrum permet aux développeurs de décider de ce qui sera livré.	Les développeurs décident uniquement de la manière de délivrer. Il revient au Product Owner de déterminer ce qui sera livré.



LES DÉVELOPPEURS

- LES DÉVELOPPEURS SONT LES MEMBRES DE L'ÉQUIPE SCRUM QUI S'ENGAGENT À TRAITER N'IMPORTE QUEL ASPECT UTILE POUR UN INCRÉMENT DURANT CHAQUE SPRINT
- LES COMPÉTENCES SPÉCIFIQUES REQUISES PAR LES DÉVELOPPEURS SONT SOUVENT LARGES ET VARIENT SELON LE DOMAINE DE TRAVAIL.
- TOUTEFOIS, LES DÉVELOPPEURS SONT TOUJOURS RESPONSABLES DE :
 - Créer un plan de Sprint : un **Sprint Backlog**
 - Inculquer la notion de **qualité** en adhérant à une **Definition of Done**
 - **Adapter leur plan** chaque jour par rapport à l'Objectif de Sprint
 - Se tenir **mutuellement responsables** en tant que professionnels



EXERCICE – DÉVELOPPEUR COMPÉTENCES

- VOUS CHERCHEZ À CONSTITUER UNE ÉQUIPE DE DÉVELOPPEMENT AGILE
- PAR GROUPE DE 2 OU 3, IDENTIFIEZ 5 QUALITÉS REQUISES POUR UN BON DÉVELOPPEUR AGILE.
- CLASSEZ-LES PAR ORDRE D'IMPORTANCE



DEVELOPMENT TEAM

- **LES PERSONNES QUI COLLABORENT**
- **LES PERSONNES QUI DEMANDENT DE L'AIDE**
- **LES PERSONNES QUI SONT PRÊTES À FAIRE DE PETITS PAS ET À OBTENIR UN RETOUR D'INFORMATION**
- **LES PERSONNES QUI SONT PRÊTES À FAIRE QUELQUE CHOSE QUI EST SUFFISANT POUR LE MOMENT.**
- **LES PERSONNES ADAPTABLES**
- **PERSONNES PRÊTES À TRAVAILLER EN DEHORS DE LEURS COMPÉTENCES**

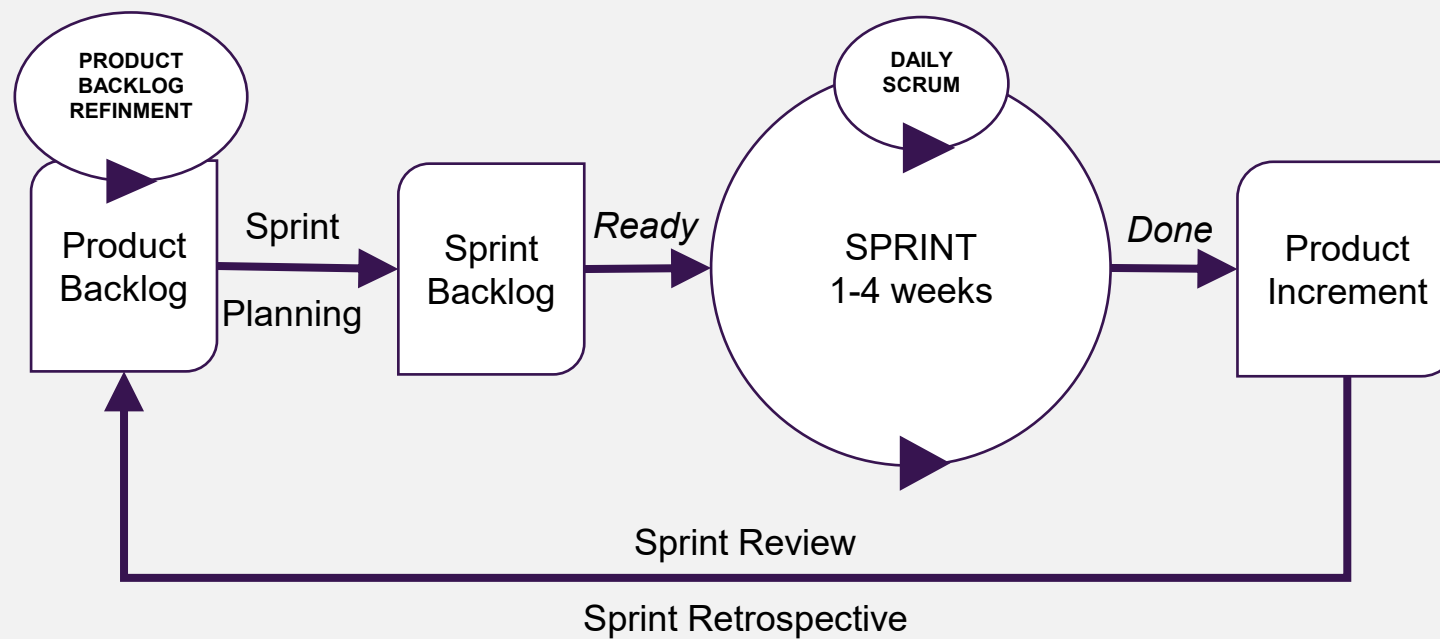
LES RITUELS





EXERCICE - SCRUM CYCLE

➤ PAR ÉQUIPE DE 3 À 5 PERSONNES, ÉBAUCHE DU CYCLE SCRUM





LES ÉVÈNEMENTS

- LES ÉVÈNEMENTS SONT UTILISÉS DANS SCRUM POUR CRÉER LA **RÉGULARITÉ** ET MINIMISER LE BESOIN DE RÉUNIONS NON DÉFINIES PAR SCRUM.
 - Idéalement, tous les événements se tiennent à la même heure et au même lieu pour réduire la complexité.
- CES ÉVÈNEMENTS SONT SPÉCIFIQUEMENT CONÇUS POUR PERMETTRE LA **TRANSPARENCE** REQUISE.
- CHAQUE ÉVÈNEMENT DANS SCRUM EST UNE OCCASION FORMELLE **D'INSPECTER ET D'ADAPTER LES ARTEFACTS** SCRUM.
 - Ne pas faire fonctionner les événements comme prescrit constitue une occasion perdue d'inspection et d'adaptation.



LE SPRINT

- **LES SPRINTS MODULENT LE RYTHME DE SCRUM, LÀ OÙ LES IDÉES SONT TRANSFORMÉES EN VALEUR.**
- **CE SONT DES ÉVÈNEMENTS D'UNE DURÉE FIXE**
 - Un mois ou moins pour créer une cohérence.
- **UN NOUVEAU SPRINT COMMENCE IMMÉDIATEMENT APRÈS LA FIN DU SPRINT PRÉCÉDENT.**
- **TOUT LE TRAVAIL NÉCESSAIRE POUR ATTEINDRE L'OBJECTIF DE PRODUIT SE FAIT DANS LE CADRE DES SPRINTS.**
- **PENDANT LE SPRINT :**
 - Les changements remettant en cause l'Objectif de sprint ne sont pas permis
 - Les objectifs de qualité sont maintenus, ils ne sont jamais revus à la baisse
 - Le Product Backlog est affiné si nécessaire
 - Le périmètre peut être clarifié et renégocié avec le Product Owner selon ce que l'équipe Scrum apprend
- **UN SPRINT PEUT ÊTRE ANNULÉ SI L'OBJECTIF DE SPRINT DEVIENT OBSOLÈTE. SEUL LE PRODUCT OWNER A LE POUVOIR D'ANNULER LE SPRINT.**



LA DURÉE DU SPRINT

- **LES SPRINTS PERMETTENT LA PRÉDICTIBILITÉ EN ASSURANT L'INSPECTION ET L'ADAPTATION DE LA PROGRESSION VERS L'OBJECTIF DE PRODUIT AU MOINS TOUS LES MOIS CALENDAIRES.**

- **RISQUES LORSQUE L'HORIZON D'UN SPRINT EST TROP LONG ?**
 - l'Objectif de Sprint peut devenir invalide
 - La complexité peut augmenter
 - Le risque peut s'accroître.

- **POURQUOI DES SPRINTS PLUS COURTS ?**
 - Pour générer plus de cycles d'apprentissage
 - Limiter le risque de coûts et d'efforts à un délai plus court

- **CHAQUE SPRINT PEUT ÊTRE CONSIDÉRÉ COMME UN PROJET COURT.**



LE CONCEPT DE TIME-BOX



- **LES BOÎTES DE TEMPS (TIME-BOX) SONT UN CONCEPT ESSENTIEL DANS SCRUM**
 - C'est une façon de rester concentré et de faire avancer les choses dans un environnement en constante évolution.
- **UNE BOÎTE DE TEMPS EST UNE PÉRIODE DE TEMPS FIXE PENDANT LAQUELLE NOUS TRAVAILLONS EN NOUS CONCENTRANT SUR CERTAINES TÂCHES OU CERTAINS OBJECTIFS**
- **TOUS LES ÉVÉNEMENTS SCRUM SONT TIME-BOXÉS, DE SORTE QUE CHAQUE ÉVÉNEMENT A UNE DURÉE MAXIMALE.**
- **NOUS SOMMES LIBRES DE MODIFIER LA DURÉE EN FONCTION DES LEÇONS APPRISES, MAIS PAS FRÉQUEMMENT ET JAMAIS EN FONCTION D'UNE OCCASION UNIQUE.**
 - Exemple : Nous ne sommes pas autorisés à dire que « nous avons beaucoup à faire cette fois-ci, alors augmentons la durée de cette cérémonie pour ce cas particulier ».
 - Ce que nous avons le droit de dire, c'est que « sur la base des dix boîtes de temps précédentes, nous nous sommes rendu compte que la durée de nos boîtes de temps ne convenait pas et qu'une augmentation de 30% de la durée pourrait mieux répondre à nos besoins ».



SPRINT PLANNING

- **OBJECTIF : INITIER LE SPRINT EN PRÉSENTANT LE TRAVAIL À EFFECTUER POUR CE SPRINT**
 - Ce plan résultant est créé par un travail collaboratif de toute la Scrum Team.
- **PARTICIPANTS : SCRUM TEAM**
 - L'équipe peut également inviter d'autres personnes à participer au Sprint Planning pour donner des conseils.
- **LE PRODUCT OWNER VEILLE À CE QUE LES PARTICIPANTS SOIENT PRÊTS**
 - A discuter des éléments du Product Backlog les plus importants
 - De la manière dont ils correspondent à l'Objectif de Produit.
- **DÉROULEMENT : LE SPRINT PLANNING ABORDE LES THÈMES SUIVANTS**
 - Thème 1 : Pourquoi ce Sprint est-il important ?
 - Thème 2 : Que peut-on faire durant ce Sprint ?
 - Thème 3 : Comment le travail choisi sera-t-il réalisé ?
- **L'OBJECTIF DE SPRINT, LES ÉLÉMENTS CHOISIS DU PRODUCT BACKLOG POUR LE SPRINT, AINSI QUE LE PLAN POUR LES LIVRER SONT APPELÉS ENSEMBLE LE **SPRINT BACKLOG**.**
- **TIMEBOX : 8 HEURES POUR UN SPRINT D'UN MOIS**



THÈME 1 : POURQUOI CE SPRINT EST-IL IMPORTANT ?

- **LE PRODUCT OWNER PROPOSE COMMENT LE PRODUIT POURRAIT AUGMENTER SA VALEUR ET SON UTILITÉ DURANT LE SPRINT ACTUEL.**
- **L'ENSEMBLE DE L'ÉQUIPE SCRUM COLLABORE ENSUITE À LA DÉFINITION D'UN OBJECTIF DE SPRINT**
 - L'Objectif de Sprint doit être finalisé avant la fin du Sprint Planning.
- **CET OBJECTIF COMMUNIQUE AUX PARTIES PRENANTES LES RAISONS POUR LESQUELLES LE SPRINT EST UTILE.**



THÈME 2 : QUE PEUT-ON FAIRE DURANT CE SPRINT ?

- EN DISCUTANT AVEC LE PRODUCT OWNER, LES DÉVELOPPEURS CHOISISSENT DES ÉLÉMENTS DU PRODUCT BACKLOG À INCLURE DANS LE SPRINT EN COURS.
 - L'équipe Scrum peut affiner ces éléments au cours de ce processus, ce qui renforce la compréhension et la confiance.

- IL PEUT ÊTRE DIFFICILE DE DÉTERMINER LA QUANTITÉ DE TRAVAIL QUI PEUT ÊTRE EFFECTUÉE DURANT UN SPRINT.
 - Cependant, plus les Développeurs en savent sur leurs performances passées, leurs capacités à venir et leur **Definition of Done**, plus ils seront confiants dans leurs prévisions de Sprint.



THÈME 3 : COMMENT LE TRAVAIL CHOISI SERA-T-IL RÉALISÉ ?

- POUR CHAQUE ÉLÉMENT CHOISI DU PRODUCT BACKLOG, LES DÉVELOPPEURS PLANIFIENT LE TRAVAIL NÉCESSAIRE POUR CRÉER UN INCRÉMENT QUI RÉPOND À LA DEFINITION OF DONE.
- CELA SE FAIT SOUVENT EN DÉCOMPOSANT LES ÉLÉMENTS DU PRODUCT BACKLOG EN ÉLÉMENTS DE TRAVAIL D'UNE JOURNÉE OU MOINS.
 - La façon de procéder est laissée à la **seule discrétion** des Développeurs.
- PERSONNE D'AUTRE NE LEUR DIT COMMENT TRANSFORMER LES ÉLÉMENTS DU PRODUCT BACKLOG EN INCRÉMENT DE VALEUR.



DAILY SCRUM

- **OBJECTIF : INSPECTER LA PROGRESSION VERS L'OBJECTIF DE SPRINT ET D'ADAPTER LE SPRINT BACKLOG SI NÉCESSAIRE, EN AJUSTANT LES FUTURS TRAVAUX PLANIFIÉS.**
 - Les Daily Scrums améliorent la communication, identifient les obstacles, favorisent la prise de décision rapide et, par conséquent, éliminent la nécessité de faire d'autres réunions.
- **PARTICIPANTS : DÉVELOPPEURS DE LA SCRUM TEAM.**
 - Si le Product Owner ou Scrum Master travaille activement sur des éléments du Sprint Backlog, ils participent en tant que Développeurs.
- **FORMAT : LES DÉVELOPPEURS PEUVENT SÉLECTIONNER LA STRUCTURE ET LES TECHNIQUES DE LEUR CHOIX**
 - A condition que leur Daily Scrum se focalise sur la progression vers l'Objectif de Sprint et produise un plan d'action pour la prochaine journée de travail.
 - Cela permet de se focaliser et d'améliorer l'autogestion.
- **TIMEBOX : 15 MINUTES QUELLE QUE SOIT LA TAILLE DE L'ÉQUIPE**
 - Pour réduire la complexité, il est tenu à la même heure et au même lieu chaque jour ouvré du Sprint.

LE DAILY SCRUM N'EST PAS LE SEUL MOMENT OÙ LES DÉVELOPPEURS SONT AUTORISÉS À AJUSTER LEUR PLAN



SPRINT REVIEW

➤ OBJECTIF : INSPECTER LE RÉSULTAT DU SPRINT ET DE DÉTERMINER LES ADAPTATIONS FUTURES.

- La Scrum Team présente les résultats de son travail aux principales parties prenantes et les progressions vers l'Objectif de Produit sont discutés.

➤ PARTICIPANTS :

- Scrum Team
- Parties prenantes

➤ DÉROULEMENT :

- La Scrum Team et les parties prenantes passent en revue ce qui a été accompli durant le Sprint et ce qui a changé dans leur environnement.
- Sur la base de ces informations, les participants collaborent sur la marche à suivre.
- Le Product Backlog peut également être ajusté pour répondre à de nouvelles opportunités.

➤ LA SPRINT REVIEW EST UNE SESSION DE TRAVAIL

- La Scrum Team doit éviter de la limiter à une session de présentation.

➤ TIMEBOX : 4 HEURES MAXIMUM POUR UN SPRINT D'1 MOIS

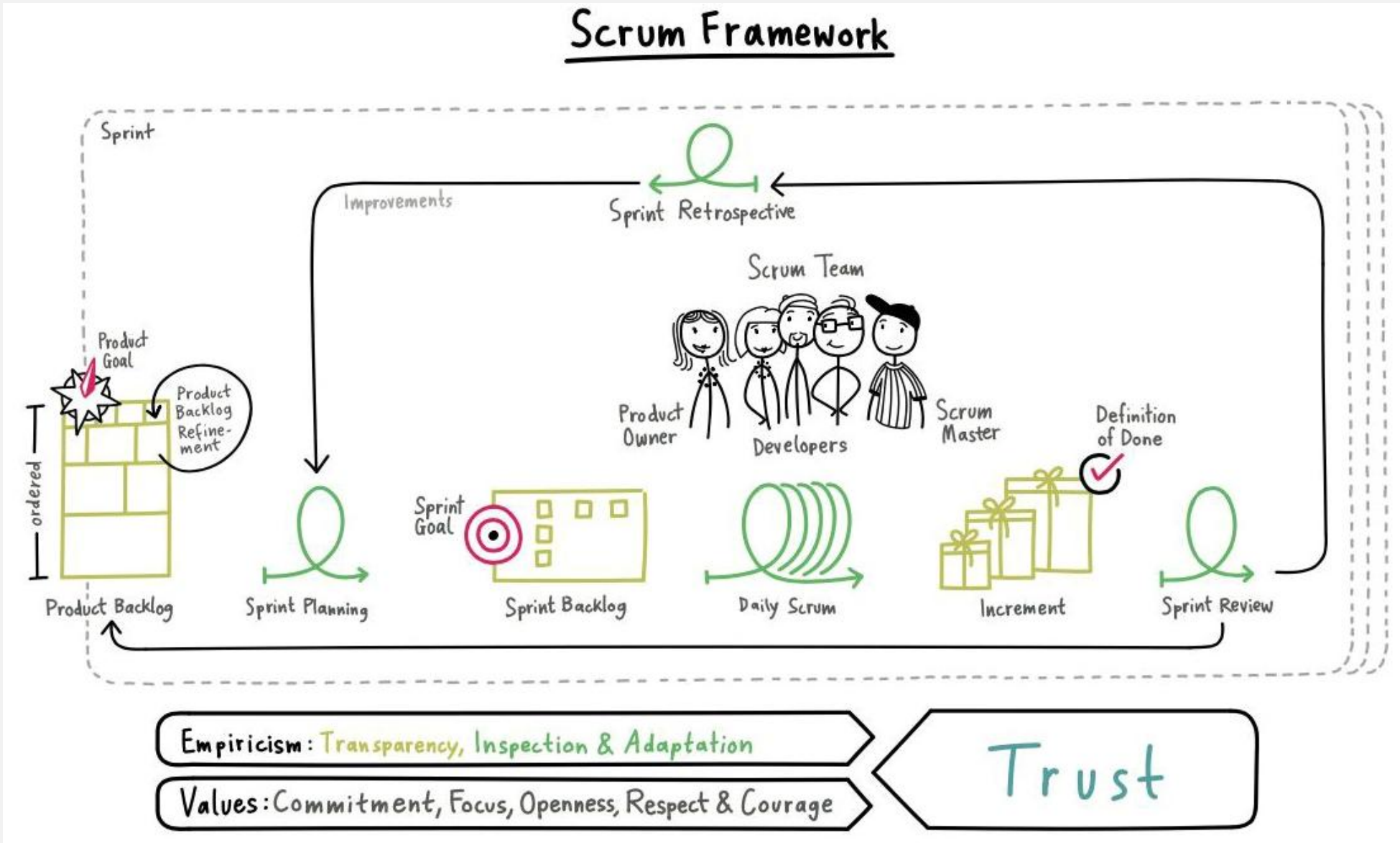


SPRINT RETROSPECTIVE

- **OBJECTIF : PLANIFIER DES MOYENS ET MÉTHODES PERMETTANT D'ACCROÎTRE LA QUALITÉ ET L'EFFICACITÉ**
- **PARTICIPANTS : SCRUM TEAM**
- **ACTIVITÉS :**
 - Inspecter le déroulement du dernier Sprint en ce qui concerne les individus, les interactions, les processus, les outils et leur Definition of Done.
 - Les hypothèses qui les ont fait dévier sont identifiées et leurs origines explorées.
- **DÉROULEMENT :**
 - La scrum Team discute de :
 - ce qui s'est bien passé pendant le Sprint
 - des problèmes rencontrés
 - de la manière dont ces problèmes ont été (ou n'ont pas été) résolus.
 - La Scrum Team identifie les changements les plus utiles pour améliorer son efficacité.
 - Les améliorations ayant le plus d'impact sont abordées dès que possible.
 - Elles peuvent même être ajoutées au Sprint Backlog pour le prochain Sprint.
- **TIMEBOX : 3 HEURES MAXIMUM POUR UN SPRINT D'1 MOIS**

UN SPRINT

OFFICIEL





EXEMPLE D'UN SPRINT D'UNE SEMAINE

Lundi	Mardi	Mercredi	Jeudi	Vendredi
Sprint Planning (1h)	Daily SCRUM (15 min)	Daily SCRUM (15 min)	Daily SCRUM (15 min)	Daily SCRUM (15 min)
		Product Backlog Refinement (1h)		Sprint Review (1h)
				Sprint Retrospective (45 min)



4 RITUELS

CHAQUE ÉVÉNEMENT EST UNE OCCASION FORMELLE D'INSPECTER ET D'ADAPTER QUELQUE CHOSE.

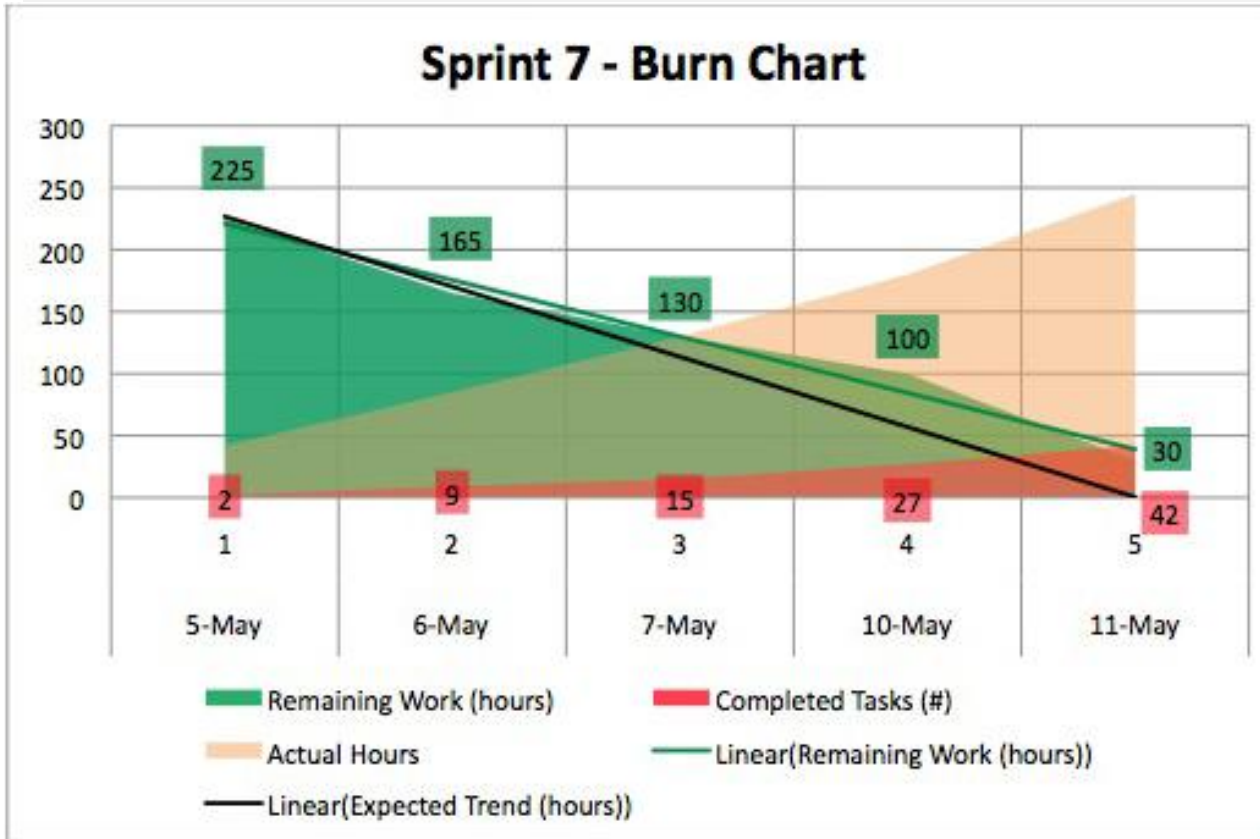
EVÈNEMENT	INSPECTION	ADAPTATION
SPRINT PLANNING	• Product Backlog • (Engagements de la rétrospective) • (Definition of Done)	• Objectif de sprint • Prévisions • Sprint Backlog
DAILY SCRUM	• Progrès vers l'objectif de sprint	• Sprint Backlog • Plan de la journée
SPRINT REVIEW	• Product Increment • Product Backlog • Conditions du marché	• Product Backlog
SPRINT RETROSPECTIVE	• Equipe et collaboration • Technologie et outils • Definition of Done	• Actions d'amélioration



RITUELS

Evènement	Timebox	Participants
Sprint planning	< 8H	Scrum Master / Product Owner / Developers
Daily Scrum	15 minutes	Developers (Product Owner / Scrum Master)
Sprint review	< 4h	Scrum Master / Product Owner / Developers / Stakeholders
Sprint retrospective	< 3h	Scrum Master / Product Owner / Developers

EXAMPLE KPI : BURNDOWN CHART



Based on "Sprint 7 Burn-Down Chart" would you do anything different in Sprint 8 ? (Choose 2 answers)

1. The Scrum Master adds additional developers for Sprint 8.
2. The Development Team re-estimates incomplete Product Backlog Items and puts them back into the Product Backlog for re-ordering. The Product Owner may ask the Development Team to complete the unfinished Product Backlog Items from Sprint 7 in Sprint 8.
3. There is nothing wrong. The Development Team will present all Product Backlog Items selected for Sprint 7 at the end of the Sprint.
4. The Development Team may forecast less overall work in Sprint 8.
5. Stakeholders will encourage the Development Team to estimate better during the Sprint 8 Planning Meeting
6. The Development Team carries over incomplete Sprint Backlog Items from Sprint 7 to Sprint 8 and monitors the Sprint 8 burn-down chart. As soon as deviation from trends is detected, the Development Team works with the Product Owner to negotiate remaining work.



CERTIFICATION CHECKPOINT

- **WHICH TOPICS SHOULD BE DISCUSSED IN THE SPRINT REVIEW ?**
- A. The Scrum process, and how it was used during the Sprint.
 - B. Coding and engineering practices
 - C. Sprint results
 - D. All of the above

VERSION TRADUITE

- **QUELS SUJETS DEVRAIENT ÊTRE DISCUTÉS DANS LA SPRINT REVIEW ?**
- A. Le processus Scrum et son utilisation pendant le sprint.
 - B. Les Pratiques de codage et d'ingénierie
 - C. Les Résultats du sprint
 - D. Tout ce qui précède



CERTIFICATION CHECKPOINT

➤ **A SPRINT RETROSPECTIVE SHOULD BE HELD**

- A. At the end of each Sprint
- B. At the beginning of each Sprint
- C. Only when the Scrum Team determines it needs one
- D. At the end of the last Sprint in a project or a release

VERSION TRADUITE

➤ **UNE RÉTROSPECTIVE DE SPRINT DEVRAIT AVOIR LIEU :**

- A. À la fin de chaque sprint
- B. Au début de chaque sprint
- C. Ce n'est que lorsque l'équipe Scrum en a besoin
- D. À la fin du dernier sprint d'un projet ou d'une version

LES ARTEFACTS





LISTE DES ARTEFACTS

➤ **PRODUCT BACKLOG**

Objectif de Produit

➤ **SPRINT BACKLOG**

Objectif de sprint

➤ **INCRÉMENT**

Definition of « Done »

LES ARTEFACTS DE SCRUM REPRÉSENTENT UN TRAVAIL OU UNE VALEUR.

CHAQUE ARTEFACT CONTIENT UN ENGAGEMENT ASSURANT LA FOURNITURE DES INFORMATIONS QUI AMÉLIORENT LA TRANSPARENCE ET LE FOCUS PAR RAPPORT AUXQUELLES LES PROGRESSIONS PEUVENT ÊTRE MESURÉES



OBJECTIF DE PRODUIT

PRODUCT GOAL

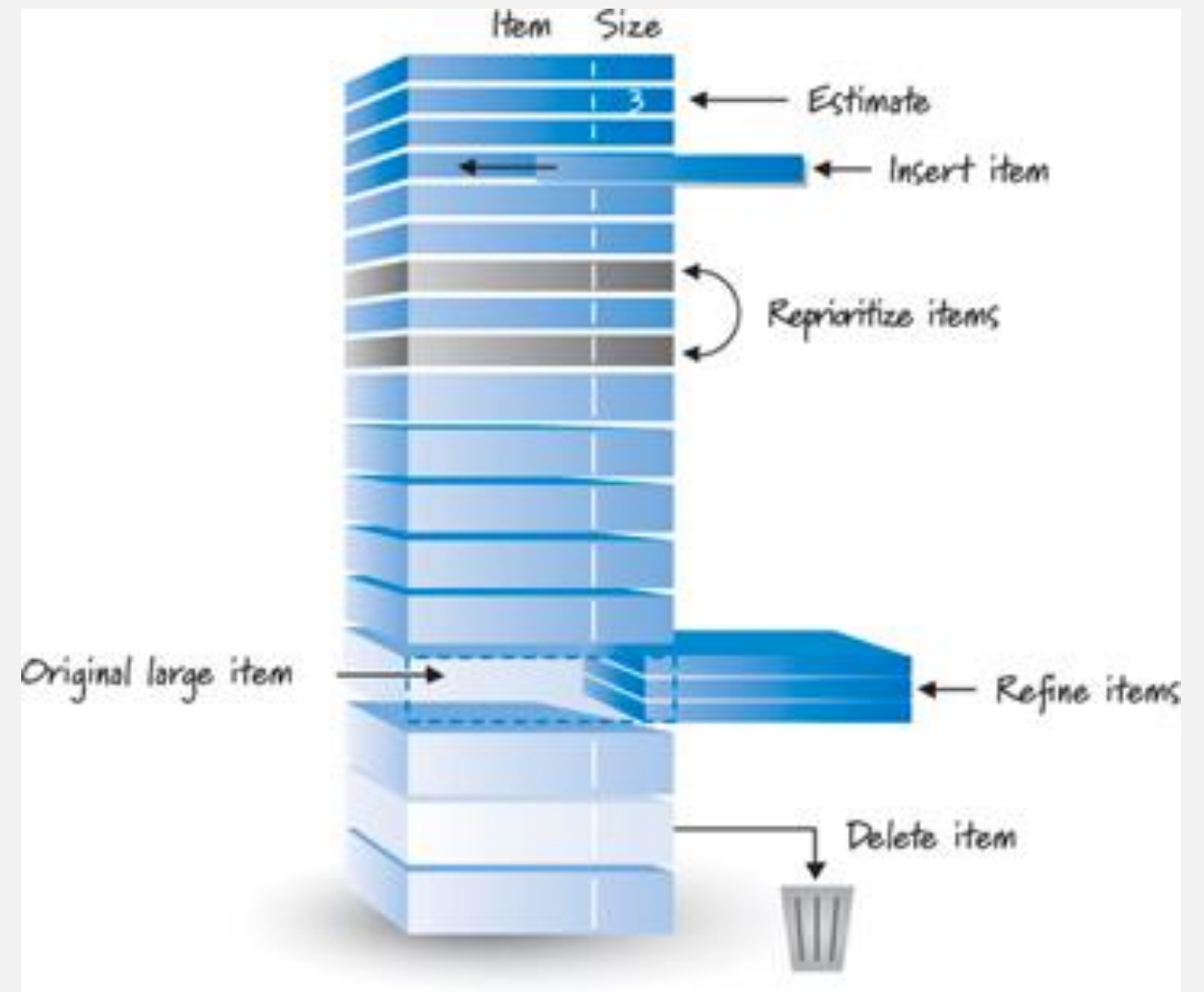
- L'OBJECTIF DE PRODUIT DÉCRIT UN ÉTAT FUTUR DU PRODUIT QUI PEUT SERVIR DE CIBLE À L'ÉQUIPE SCRUM POUR PLANIFIER.
- C'EST UN OBJECTIF À LONG TERME DE L'ÉQUIPE DE SCRUM. ILS DOIVENT ATTEINDRE UN OBJECTIF AVANT DE S'ATTAQUER AU SUIVANT.



UN PRODUIT EST UN MOYEN DE CRÉER DE LA VALEUR. IL A UNE FRONTIÈRE CLAIRE, DES PARTIES PRENANTES CONNUES, DES UTILISATEURS OU DES CLIENTS BIEN DÉFINIS. UN PRODUIT PEUT ÊTRE UN SERVICE, UN PRODUIT PHYSIQUE OU QUELQUE CHOSE PLUS ABSTRAIT.

PRODUCT BACKLOG

- LE PRODUCT BACKLOG EST UNE LISTE ORDONNÉE ET ÉMERGENTE DE CE QUI EST NÉCESSAIRE POUR AMÉLIORER LE PRODUIT.
- C'EST LA SEULE SOURCE DU TRAVAIL ACCOMPLI PAR L'ÉQUIPE SCRUM.





AFFINEMENT DU PRODUCT BACKLOG

- **L'AFFINEMENT DU PRODUCT BACKLOG CONSISTE À DÉCOMPOSER ET À DÉFINIR DAVANTAGE LES ÉLÉMENTS DU BACKLOG EN ÉLÉMENTS PLUS FINS ET PLUS PRÉCIS.**

- **IL S'AGIT D'UNE ACTIVITÉ CONTINUE VISANT À AJOUTER DES DÉTAILS :**
 - Tels qu'une description, un ordre et une taille.
 - (Les attributs varient souvent en fonction du domaine de travail)



RESPONSABILITÉ

➤ LE PRODUCT OWNER EST RESPONSABLE DE LA GESTION EFFICACE DU PRODUCT BACKLOG, QUI COMPREND :

- Développer et communiquer explicitement l'objectif de produit
- Créer et communiquer clairement les éléments du product backlog
- Ordonner les éléments dans le product backlog ; et,
- S'assurer que le product backlog est transparent, visible et compréhensible.

➤ LES DÉVELOPPEURS QUI EFFECTUERONT LE TRAVAIL SONT RESPONSABLES DU DIMENSIONNEMENT.

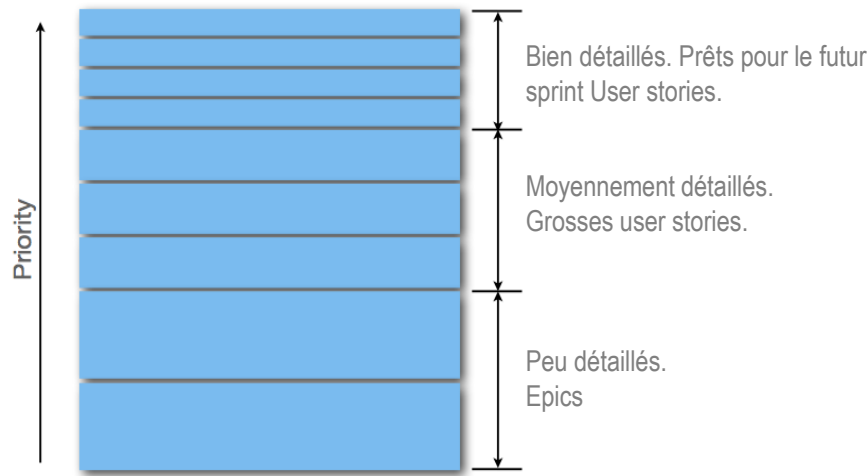
➤ LE PRODUCT OWNER PEUT INFLUENCER LES DÉVELOPPEURS EN LES AIDANT À COMPRENDRE ET TROUVER DES COMPROMIS.



NOTION DE PRÊT

DEFINITION OF READY (DOR)

- **LES ÉLÉMENTS DU PRODUCT BACKLOG QUI PEUVENT ÊTRE TRAITÉS PAR L'ÉQUIPE SCRUM DURANT UN SPRINT SONT CONSIDÉRÉS COMME PRÊTS POUR LA SÉLECTION DANS UN ÉVÉNEMENT DE SPRINT PLANNING.**
- **ILS ACQUIÈRENT GÉNÉRALEMENT CE DEGRÉ DE TRANSPARENCE APRÈS AVOIR ÉTÉ AFFINÉS.**

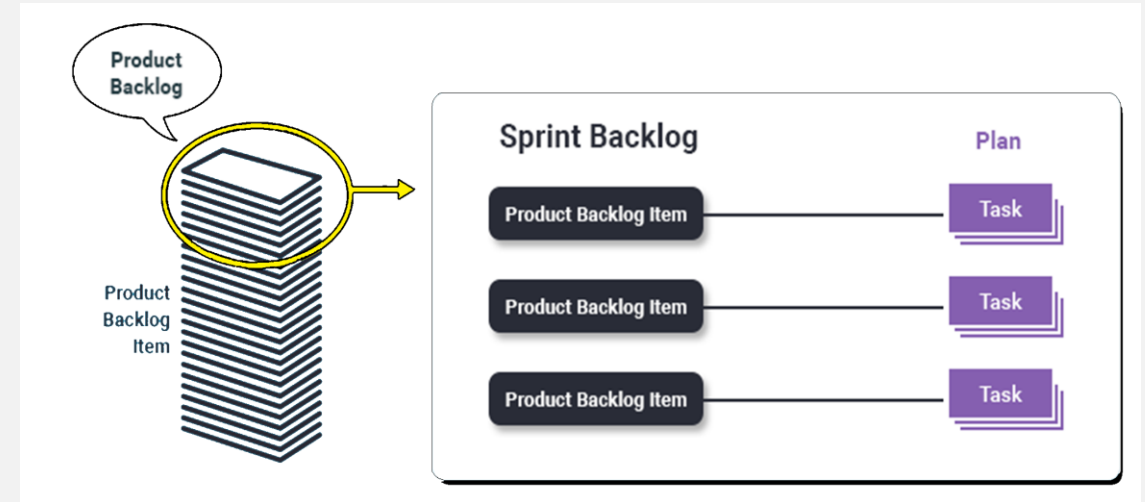


Exemple d'une DOR :

- ☐ User Story définie
- ☐ Critères d'acceptation définis
- ☐ Dépendances identifiées
- ☐ User Story estimée par l'équipe
- ☐ Scrum Team accepte l'IHM
- ☐ Critères de performance identifiés, quand cela est approprié
- ☐ L'équipe a une bonne idée de ce qui sera démontré



SPRINT BACKLOG



➤ LE SPRINT BACKLOG EST COMPOSÉ DE :

- L'OBJECTIF DE SPRINT (POURQUOI)
- L'ENSEMBLE DES ÉLÉMENTS DU PRODUCT BACKLOG CHOISIS POUR LE SPRINT (QUOI)
- UN PLAN D'ACTION POUR LA RÉALISATION DE L'INCRÉMENT (COMMENT).

➤ LE SPRINT BACKLOG EST UN PLAN ÉLABORÉ PAR ET POUR LES DÉVELOPPEURS.

➤ IL S'AGIT D'UNE IMAGE TRÈS VISIBLE ET EN TEMPS RÉEL DU TRAVAIL QUE LES DÉVELOPPEURS PRÉVOIENT D'ACCOMPLIR DURANT LE SPRINT AFIN D'ATTEINDRE L'OBJECTIF DE SPRINT.

- Par conséquent, le sprint backlog est mis à jour tout au long du sprint selon ce qu'on en apprend.

➤ IL DOIT ÊTRE SUFFISAMMENT DÉTAILLÉ POUR QUE LES DÉVELOPPEURS PUISSENT INSPECTER LEUR PROGRESSION DURANT LA MÊLÉE QUOTIDIENNE.



OBJECTIF DE SPRINT

L'OBJECTIF DE SPRINT EST LE SEUL BUT DU SPRINT



- BIEN QUE L'OBJECTIF DE SPRINT SOIT UN ENGAGEMENT FAIT PAR LES DÉVELOPPEURS, IL OFFRE UNE CERTAINE FLEXIBILITÉ EN TERMES DE TRAVAIL NÉCESSAIRE POUR ATTEINDRE CET OBJECTIF.
- L'OBJECTIF DE SPRINT CRÉE ÉGALEMENT DE LA COHÉRENCE ET DU FOCUS, TOUT EN ENCOURAGEANT L'ÉQUIPE SCRUM À TRAVAILLER ENSEMBLE PLUTÔT QUE SUR DES INITIATIVES SÉPARÉES.
- L'OBJECTIF DE SPRINT EST CRÉÉ LORS DE L'ÉVÈNEMENT DE SPRINT PLANNING, PUIS AJOUTÉ AU SPRINT BACKLOG. PENDANT QUE LES DÉVELOPPEURS TRAVAILLENT DURANT LE SPRINT, ILS GARDENT L'OBJECTIF DE SPRINT À L'ESPRIT.
- SI LE TRAVAIL S'AVÈRE ÊTRE DIFFÉRENT DE CE À QUOI LES DÉVELOPPEURS S'ATTENDAIENT, ILS COLLABORENT AVEC LE PRODUCT OWNER POUR NÉGOCIER LE PÉRIMÈTRE DU SPRINT BACKLOG AU SEIN DU SPRINT SANS AFFECTER L'OBJECTIF DE SPRINT.

INCRÉMENT

- UN INCRÉMENT EST UNE PREMIÈRE ÉTAPE CONCRÈTE VERS L'OBJECTIF DE PRODUIT.
- CHAQUE INCRÉMENT S'AJOUTE À TOUS LES INCRÉMENTS PRÉCÉDENTS ET FAIT L'OBJET D'UNE VÉRIFICATION APPROFONDIE, CE QUI GARANTIT QUE TOUS LES INCRÉMENTS FONCTIONNENT ENSEMBLE.
 - Afin de fournir une valeur, l'incrément doit être utilisable.
- PLUSIEURS INCRÉMENTS PEUVENT ÊTRE CRÉÉS DURANT UN SPRINT. LA SOMME DES INCRÉMENTS EST PRÉSENTÉE LORS DE LA SPRINT REVIEW, CE QUI PERMET AINSI DE SOUTENIR L'EMPIRISME.
- LE TRAVAIL NE PEUT ÊTRE CONSIDÉRÉ COMME FAISANT PARTIE D'UN INCRÉMENT QUE S'IL SATISFAIT À LA DÉFINITION DE FINI.





DÉFINITION DE FINI

DEFINITION OF DONE

- **LA DÉFINITION DE FINI EST UNE DESCRIPTION FORMELLE DE L'ÉTAT DE L'INCRÉMENT LORSQU'IL SATISFAIT AUX MESURES DE QUALITÉ REQUISES POUR LE PRODUIT.**
- **LA DÉFINITION DE FINI CRÉE DE LA TRANSPARENCE EN PERMETTANT À CHACUN DE COMPRENDRE LE TRAVAIL ACCOMPLI DANS LE CADRE DE L'INCRÉMENT.**
- **SI UN ÉLÉMENT DU PRODUCT BACKLOG N'EST PAS CONFORME À LA DÉFINITION DE FINI, IL NE PEUT PAS ÊTRE PUBLIÉ NI MÊME PRÉSENTÉ LORS DE LA SPRINT REVIEW.**

Il est alors renvoyé au Product Backlog pour un examen ultérieur.

Exemple d'une DOD :

- ☐ Version potentiellement livrable disponible en téléchargement
- ☐ Résumé des modifications mis à jour pour inclure les nouvelles fonctionnalités implémentées
- ☐ Fonctionnalités inactives / non implémentées masquées ou grisées (non exécutables)
- ☐ Tests unitaires écrits et verts
- ☐ Code source validé sur le serveur
- ☐ L'Intégration Continue construit la version et tous les tests en vert
- ☐ Révision du code terminée (ou programmée par paire)
- ☐ Validation PO OK



DEFINITION OF DONE DANS L'ORGANISATION

- SI LA *DEFINITION OF DONE* POUR UN *INCREMENT* FAIT PARTIE DES STANDARDS DE L'ORGANISATION, TOUTES LES SCRUM TEAMS DOIVENT LA SUIVRE AU MINIMUM.
- S'IL NE S'AGIT PAS D'UNE NORME DE L'ORGANISATION, L'ÉQUIPE SCRUM DOIT CRÉER UNE *DEFINITION OF DONE* APPROPRIÉE POUR LE PRODUIT.
- LES DÉVELOPPEURS SONT TENUS DE SE CONFORMER À LA *DEFINITION OF DONE*. SI PLUSIEURS SCRUM TEAMS TRAVAILLENT ENSEMBLE SUR UN PRODUIT, ELLES DOIVENT DÉFINIR MUTUELLEMENT ET SE CONFORMER À LA MÊME *DEFINITION OF DONE*.



CERTIFICATION CHECKPOINT

➤ HOW SMALL SHOULD A 'READY' PRODUCT BACKLOG ITEM BE?

- A. Small enough to build in 1 Sprint
- B. No longer than one day
- C. Small enough for a single team member to complete in a Sprint.
- D. No bigger than 8 story points.
- E. It should fit on a small index card.

VERSION TRADUITE

➤ QUELLE DOIT ÊTRE LA TAILLE D'UN ÉLÉMENT DU BACKLOG DE PRODUIT « PRÊT » ?

- A. Assez petit pour être construit en 1 Sprint.
- B. Pas plus d'un jour.
- C. Assez petit pour qu'un seul membre de l'équipe puisse LE TERMINER EN un sprint.
- D. Pas plus grand que 8 STORY POINTS.
- E. Il devrait tenir sur une petite fiche.



CERTIFICATION CHECKPOINT

- **HOW MUCH OF THE SPRINT BACKLOG MUST BE DEFINED DURING THE SPRINT PLANNING MEETING?**
- A. Just enough tasks for the Scrum Master to be confident in the Team's understanding of the Sprint.
 - B. Just enough to understand design and architectural implications.
 - C. All of the potential work. The Sprint Planning meeting isn't over until 100% of the work is identified and estimated.
 - D. Enough so the Team can create its best forecast of what it can do, and to start the first several days of the Sprint.

VERSION TRADUITE

- **QUELLE QUANTITÉ DU SPRINT BACKLOG DOIT ÊTRE DÉFINIE LORS DU SPRINT PLANNING ?**
- A. Suffisamment de tâches pour que le Scrum Master soit confiant dans la compréhension du Sprint par l'équipe
 - B. Juste assez pour comprendre les implications conceptuelles et architecturales.
 - C. Tous les travaux potentiels. La réunion de planification du sprint n'est pas terminée avant que 100% du travail ne soit identifié et estimé.
 - D. Assez pour que l'équipe puisse établir ses meilleures prévisions et commencer les premiers jours du Sprint.



CERTIFICATION CHECKPOINT

➤ WHICH OF THE FOLLOWING BEST DESCRIBES AN INCREMENT OF WORKING SOFTWARE?

- A. An automated test suite to verify functionality delivered in previous iterations.
- B. UML diagrams that describe how to deliver functionality in future iterations.
- C. A decomposition of all Product Backlog items into tasks for future Sprint Backlog lists.
- D. A new user interface design for functionality delivered in previous iterations.
- E. Additional features in a useable state that complement those delivered in previous iterations.

VERSION TRADUITE

➤ LEQUEL DES ÉNONCÉS SUIVANTS DÉCRIT LE MIEUX UN INCRÉMENT DE LOGICIEL FONCTIONNEL ?

- A. Une suite de tests automatisés pour vérifier les fonctionnalités fournies lors des itérations précédentes.
- B. Les diagrammes UML décrivent comment fournir des fonctionnalités dans les futures itérations.
- C. Une décomposition de tous les éléments du PRODUCT BACKLOG en tâches pour les futurs SPRINT BACKLOG
- D. Une nouvelle conception d'interface utilisateur pour les fonctionnalités fournies dans les itérations précédentes.
- E. Fonctionnalités supplémentaires dans un état utilisable qui complètent celles fournies dans les itérations précédentes.



CERTIFICATION CHECKPOINT

➤ IF BURNDOWN CHARTS ARE USED TO VISUALIZE PROGRESS, WHAT DO THEY TRACK?

- A. Accumulated cost
- B. Work remaining across time
- C. Accumulated business value delivered to the customer
- D. Individual worker productivity

VERSION TRADUITE

➤ SI LES BURNDOWN CHARTS SONT UTILISÉS POUR VISUALISER LES PROGRÈS, QUE SUIVENT-ILS ?

- A. Coût accumulé
- B. Travail restant dans le temps
- C. Valeur commerciale cumulée livrée au client
- D. Productivité individuelle des travailleurs



CERTIFICATION CHECKPOINT

➤ **WHICH TWO STATEMENTS EXPLAIN WHY THE DEFINITION OF “DONE” IS IMPORTANT TO THE PRODUCT OWNER ?**

- A. It assure the Increment reviewed at the sprint review is usable so the Product Owner may choose to release it
- B. It helps the Product Owner track the open work during the sprint
- C. It creates transparency regarding progress within the Scrum team
- D. It identifies undone work that can be addressed in a separate sprint

VERSION TRADUITE

➤ **QUELLES SONT LES DEUX DÉCLARATIONS EXPLIQUANT POURQUOI LA « DEFINITION OF DONE » EST IMPORTANTE POUR LE PRODUCT OWNER ?**

- A. Cela garantit que l'incrément examiné lors de l'examen du sprint est utilisable pour que le responsable du produit puisse choisir de le publier.
- B. Il aide le propriétaire de produit à suivre le travail ouvert pendant le sprint
- C. Cela crée de la transparence concernant les progrès au sein de l'équipe Scrum
- D. Il identifie le travail annulé qui peut être traité dans un sprint séparé

DÉVELOPPEMENT DES PERSONNES ET DES ÉQUIPES

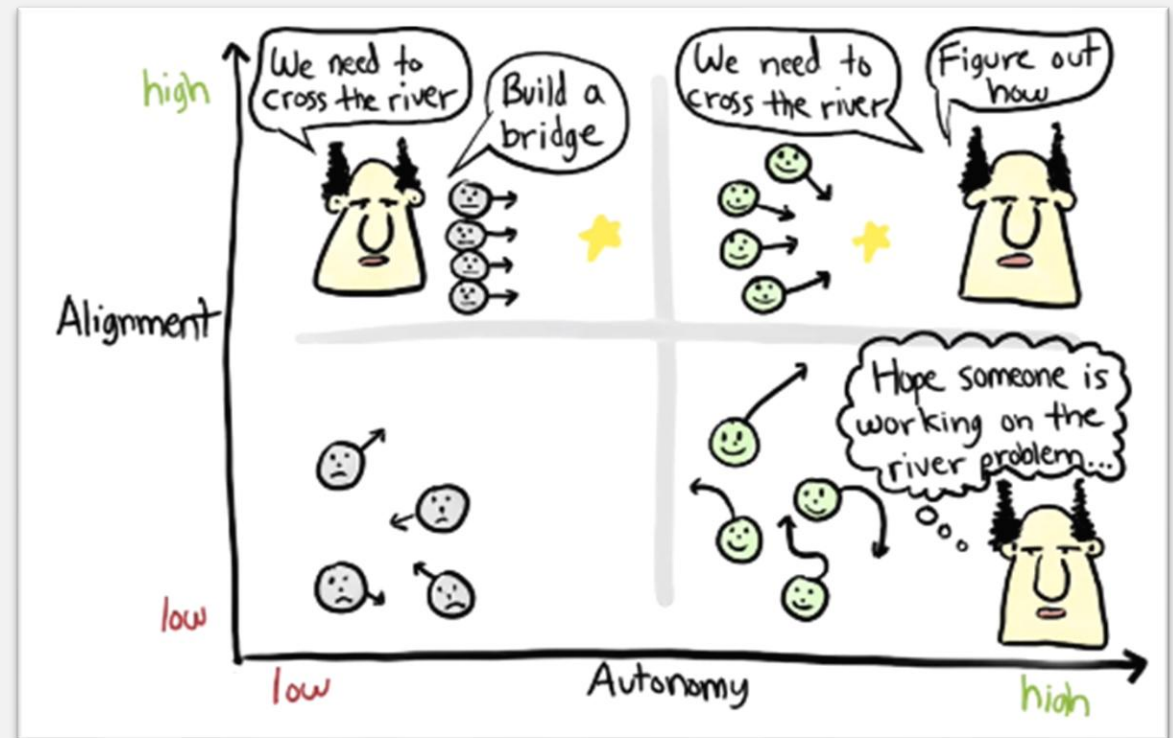




L'ÉQUIPE SCRUM EST AUTO GÉRÉE

NUL (PAS MÊME LE SCRUM MASTER) N'INDIQUE À L'ÉQUIPE DE DÉVELOPPEMENT COMMENT TRANSFORMER LES ÉLÉMENTS DU PRODUCT BACKLOG EN INCRÉMENTS DE FONCTIONNALITÉS POTENTIELLEMENT PUBLIABLES.

- Obtenir un engagement partagé, puisque l'équipe s'engage sur l'objectif et trouve les moyens ensemble pour y parvenir





QUI MET EN PLACE L'AUTOGESTION ?

➤ LE SCRUM MASTER DOIT AVOIR UNE POSTURE DE "SERVANT LEADER", POUR AIDER L'ÉQUIPE À

- Atteindre ses objectifs
- Surmonter les obstacles
- Faire les meilleurs choix (et non pas les faire à leur place)
- S'autogérer

➤ IL DOIT AUSSI AIDER L'ÉQUIPE À COMPRENDRE LES BIENFAITS DE LA NON-SPÉCIALISATION DES PROFILS

- Planifier plus facilement et plus aisément le travail à faire (congrés, maladies, affectation sur d'autres projets, etc.)
- Faire monter en compétences les personnes les moins expérimentées de l'équipe et ainsi les motiver pour qu'elles puissent évoluer et donner le meilleur d'elles-mêmes
- Éviter les situations critiques avec aucune connaissance de l'équipe du travail à faire



QUAND METTRE EN PLACE L'AUTOGESTION ?

EN FAIT, TOUT AU LONG DE LA VIE DU PROJET !

➤ L'AUTOGESTION DE L'ÉQUIPE EST UN CHANGEMENT DE PARADIGME POUR

- Les équipiers qui n'ont jamais travaillé dans un contexte agile
- Mais aussi le management / les chefs projets

➤ CE N'EST PAS INNÉ ET N'ARRIVE PAS PAR MAGIE DÈS LE DÉBUT DU PROJET

➤ IL PEUT SUFFIRE D'UN ACCIDENT DE PARCOURS (UN ÉVÉNEMENT GRAVE) POUR CASSER L'AUTOGESTION

- Exemple: livraison d'un sprint de mauvaise qualité entraînant une grosse insatisfaction client

➤ C'EST DONC À RÉALISER TOUT AU LONG DU PROJET



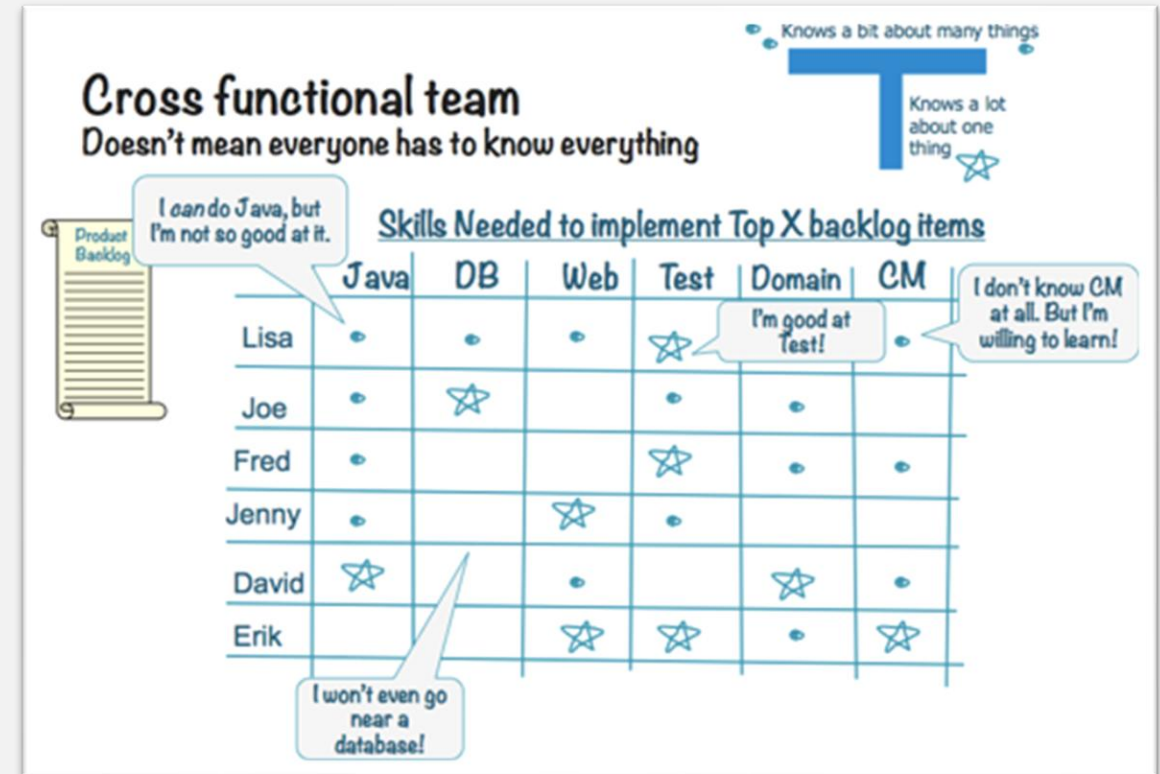
UNE ÉQUIPE SCRUM PLURI-DISCIPLINAIRE

LEURS MEMBRES ONT TOUTES LES COMPÉTENCES NÉCESSAIRES POUR CRÉER DE LA VALEUR À CHAQUE SPRINT

➤ Tous les membres de l'équipe n'ont pas besoin de savoir tout faire

- En théorie, tous les membres de l'équipe maîtrisent toutes les compétences (vision idyllique mais PSD1)

➤ Mais il faut éviter, opérationnellement, d'avoir des goulots d'étranglement autour de personnes « expertes et/ou spécialisées » dans un domaine





DEVENIR AUTONOME

- **LE RÔLE DU SCRUM MASTER N'EST PAS, DANS UN CONTEXTE AGILE, DE DIRE À L'ÉQUIPE CE QU'ELLE DOIT FAIRE, QUELLES SONT LES TÂCHES À EFFECTUER ET COMMENT LES FAIRE.**
 - En effet, dans un tel contexte, le rôle du leader est d'aider l'équipe à grandir et à murir, il doit de ce fait se positionner en Servant Leader
 - Ce n'est pas un chef de projet,
- **TOUTES LES TACHES DOIVENT ETRE ASSUMÉES PAR L'ÉQUIPE (AFFECTATION DE TACHES, EXPERTISE TECHNIQUE, TESTS...)**



RECOMMANDATIONS

A L'INTENTION DES MEMBRES D'ÉQUIPE

➤ PRENDRE DES DÉCISIONS PAR **CONSENSUS**

- L'équipe aura beaucoup de décisions à prendre, les membres devront s'accorder en équipe sur la façon dont elle les prendra.

➤ COACHING DE PAIR ET RETOUR D'INFORMATIONS

- L'équipe doit rechercher des moyens de devenir **polyvalente** et d'augmenter l'apprentissage. Une possibilité est de faire du coaching de pair et de se donner des retours d'informations les uns les autres.

➤ RÉSOUDRE LES **PROBLÈMES** EN GROUPE

- Les membres de l'équipe sont responsables de résoudre rapidement les problèmes, ensemble dès qu'ils surviennent.

➤ GÉRER LES **CONFLITS**

- Les membres devront s'accorder sur la façon dont on résout les conflits au sein de l'équipe.



QUESTIONS DE CERTIFICATION

EXEMPLES DE QUESTIONS DE LA CERTIFICATION

- **What are two responsibilities of testers in a Scrum Team ? (Choose 2 answers)**
 - A. Everyone in the Team is responsible for quality
 - B. Scrum has no "tester" role
 - C. Finding bugs
 - D. Verifying the work of programmers
 - E. Tracking quality metrics
- **Who is responsible for the system architecture of a product developed using Scrum?**
 - A. The Team
 - B. The architect chosen by the Team
 - C. The software architect assigned to the Team
 - D. The Chief Architect



QUESTIONS DE CERTIFICATION

EXEMPLES DE QUESTIONS DE LA CERTIFICATION

- **Which does a self-managed Team choose?**
 - A. Sprint length
 - B. Stakeholders for the Sprint Review
 - C. When to release, based on its progress
 - D. Product Backlog ordering
 - E. How to best accomplish its work
- **What are three benefits of self-management?**
 - A. Increased accuracy of estimates
 - B. Increased commitment
 - C. Increased creativity
 - D. Increased self-accountability
 - E. Increased rule compliance

**SQLI
DIGITAL
EXPERIENCE**

SCRUM DEV FORMATION

J2

V 3.1.0

DÉVELOPPER ET LIVRER DES PRODUITS DE MANIÈRE PROFESSIONNELLE





DÉVELOPPER ET LIVRER DES PRODUITS DE MANIÈRE PROFESSIONNELLE



Développement logiciel itératif



La qualité et Scrum



Pair-programming



Clean Code



La dette technique



Source control & CI/CD

DÉVELOPPEMENT LOGICIEL ITÉRATIF





CONCEPTION TECHNIQUE : LES OUTILS ET PRATIQUES

DÉVELOPPEMENT LOGICIEL ITÉRATIF

- Dans un contexte Agile, la conception a pour objectif d'apporter le maximum de **flexibilité** tout en restant concentré sur la **qualité**

Les développements sont alors soumis à certains enjeux ...

- **Enjeu 1 : être incrémental**
 - Les développements doivent être fait avec une **vision incrémentale** du produit ; la mise en place à **minima effectuée aujourd'hui** doit pouvoir **évoluer facilement** sans remettre en cause l'ensemble de l'application
- **Enjeu 2 : être pragmatique & automatisable**
 - Éliminer au maximum le **coût de la récurrence**
 - Permettre leur packaging/configuration/déploiement automatisé
 - Permettre leur intégration continue, etc.
 - Réfléchir au gain du **spécifique vs générique**
- **Enjeu 3 : être maintenable**
 - Concevoir des composants techniques qui peuvent être testés unitairement de manière fiable et répétable (cf. tests unitaires automatisés).
 - Pour rester agile, il ne faut faire « que ce qui est nécessaire » et ne pas s'encombrer inutilement de documentations ou processus. Cependant, il appartient à l'équipe d'identifier la **meilleure approche** pour maintenir les connaissances qui lui permettront de **maintenir** l'application (guide des bonnes pratiques, javadoc, spécifications, wiki...) et assurer la **qualité** des développements sur la durée.



CONCEPTION TECHNIQUE : LES OUTILS & PRATIQUES

DÉVELOPPEMENT LOGICIEL ITÉRATIF

- Dans un contexte Agile, il n'y a pas de phase de conception en parallèle ou amont d'une phase de développement, ni de délégation de cette tâche à une autre équipe ou expert.
- L'équipe de développement SCRUM est responsable de la conception et doit savoir proposer (au Product Owner) différentes alternatives.
- Dans la pratique, la conception technique se fait à différents moments, suivant la complexité des sujets.
 - *Simple*
 - Discuté durant le Sprint Planning, développé durant le Sprint (sous réserve d'acceptation par le Product Owner)
 - *Complexe*
 - Préparé en amont du Sprint Planning (via des ateliers « Backlog Refinement » durant le Sprint précédent) pour estimer l'effort en séance.
 - Synchronisation avec le PO
 - Décomposition en sujets simples
 - *Très complexe avec impacts et/ou risques à lever*
 - Conception via une tâche dédiée (on parle alors de « spike ») traitée sur 1 ou plusieurs sprints jusqu'à maturité de la solution proposée.
 - Nécessité de faire un POC technique pour valider une ou plusieurs pistes de travail.



CONCEPTION TECHNIQUE : LES OUTILS & PRATIQUES

DÉVELOPPEMENT LOGICIEL ITÉRATIF

BON À SAVOIR : DANS LA CERTIFICATION, SI UNE QUESTION SOUS-ENTEND QU'UNE TECHNOLOGIE, UN OUTIL OU UNE PRATIQUE PERMET DE RÉGLER UNE PROBLÉMATIQUE ORGANISATIONNELLE, ALORS IL S'AGIT D'UNE QUESTION PIÈGE.



You are working with multiple Development Teams that are working in the same code base. They suffer from coordination problems and mutual dependencies. Some team members voice the opinion that this can be fully addressed with a comprehensive branching and merging model in the version control tool.

This is: True/False

- Les méthodologies Agile n'imposent aucune pratique en termes de documentation
- Les méthodologies Agile n'imposent aucun outil
- L'équipe est donc libre de proposer et de s'auto-Gérer.
- Quelques pratiques :
 - Architecture applicative affichée au mur
 - Outils de revue de code (Gitlab, Github, Fisheye...)
 - Standards et bonnes pratiques affichées au mur
 - Wiki projet (plan de configuration, infos importantes...)
 - Ecran affichant l'état des builds
 - ...



CONCEPTION TECHNIQUE : LES OUTILS & PRATIQUES

DÉVELOPPEMENT LOGICIEL ITÉRATIF

➤ EXEMPLES DE QUESTIONS DE LA CERTIFICATION :

- Which of the following describes an architecture spike?
 1. A small development activity to learn about technical elements of a proposed solution
 2. A decision made by a systems architect to settle disagreement within a Development Team.
 3. The result of an exhaustive architectural planning effort.
 4. A fundamental architectural problem found in an existing application.
- Database design must be complete before coding starts to ensure a solid foundation.
 1. True
 2. False
- When is a system's architecture decided?
 1. Before writing code
 2. It is designed along with the vision, before the first Sprint
 3. Throughout development, as understanding emerges
 4. In the beginning of the project during Sprint 0



CONCEPTION TECHNIQUE : LES OUTILS & PRATIQUES

DÉVELOPPEMENT LOGICIEL ITÉRATIF

➤ EXEMPLES DE QUESTIONS DE LA CERTIFICATION :

- Choose two common cross-cutting concerns in application architecture. (Choose 2 answers)
 1. Build
 2. Scalability
 3. Security
 4. Design pattern selection
- How is architecture decided in a Scrum Team during a project?
 1. The architect works in the early part of each Sprint to define the architecture for that Sprint.
 2. The Development Team emerges system architecture as the system is created.
 3. The architecture cares for itself as code is written and deployed.
 4. An architect on the Development Team undertakes all architecture tasks.
 5. Upfront designs are presented to the Scrum Team prior to the first Sprint.
- What are two ways a Development Team can ensure a good application architecture? (Choose 2 answers)
 1. The Development Team assigns someone to the role of software architect to provide architectural oversight.
 2. The Development Team has a set of guiding architecture principles that every Development Team member understands and follows when writing code.
 3. The Development Team uses a Sprint 0 to create infrastructure that will be needed later.
 4. The Development Team focuses only on adding functionality. The architecture will take care of itself as the Development Team adds functionality.
 5. Architecture is an ongoing discussion in the Development Team, focusing on implementing current Sprint Backlog items.



LES PROBLÉMATIQUES LIÉES À UNE CONCEPTION AGILE

DÉVELOPPEMENT LOGICIEL ITÉRATIF

- Les développements se font par sprint ce qui enlève la vision d'ensemble.
- Le risque est d'avoir un produit avec un amoncellement d'évolution superposées les unes sur les autres sans cohérence.
- Risque de réinventer la roue.
- Risque de complexifier le code dans sa compréhension et sa maintenabilité.
- Risque d'introduire des régressions.
- A l'inverse, risque d'avoir un code trop générique sans réel besoin.

**POUR ÉVITER CES PROBLÉMATIQUES, IL FAUT PRODUIRE UN
CODE DE QUALITÉ : LE CLEAN CODE !**

LA QUALITÉ ET SCRUM





LES ENJEUX

LA QUALITÉ ET SCRUM

- **Le risque majeur est la régression technique ou fonctionnelle :**
 - « Ce qui fonctionnait avant ne fonctionne plus »
 - « Pourtant, ça marche sur mon poste... »
 - « Tiens ? Il me semble que j'avais corrigé ce bug dans la fonctionnalité similaire »
 - « Avec cette version c'est très lent.. »
 - ...
- **Le contrôle qualité est fondamental dans la mise en place de l'Agilité : l'objectif est de garantir/maintenir la qualité de l'application sur la durée malgré les cycles de développement très courts.**
- **Pour bâtir une stratégie de tests, il faut avant toute chose s'articuler autour d'une vision et un objectif.**
- **L'automatisation permet sur la durée de réduire le coût élevé d'exécution des tests et permettre ainsi leur exécution en continue.**



LES TESTS DE NON-RÉGRESSION

LA QUALITÉ ET SCRUM

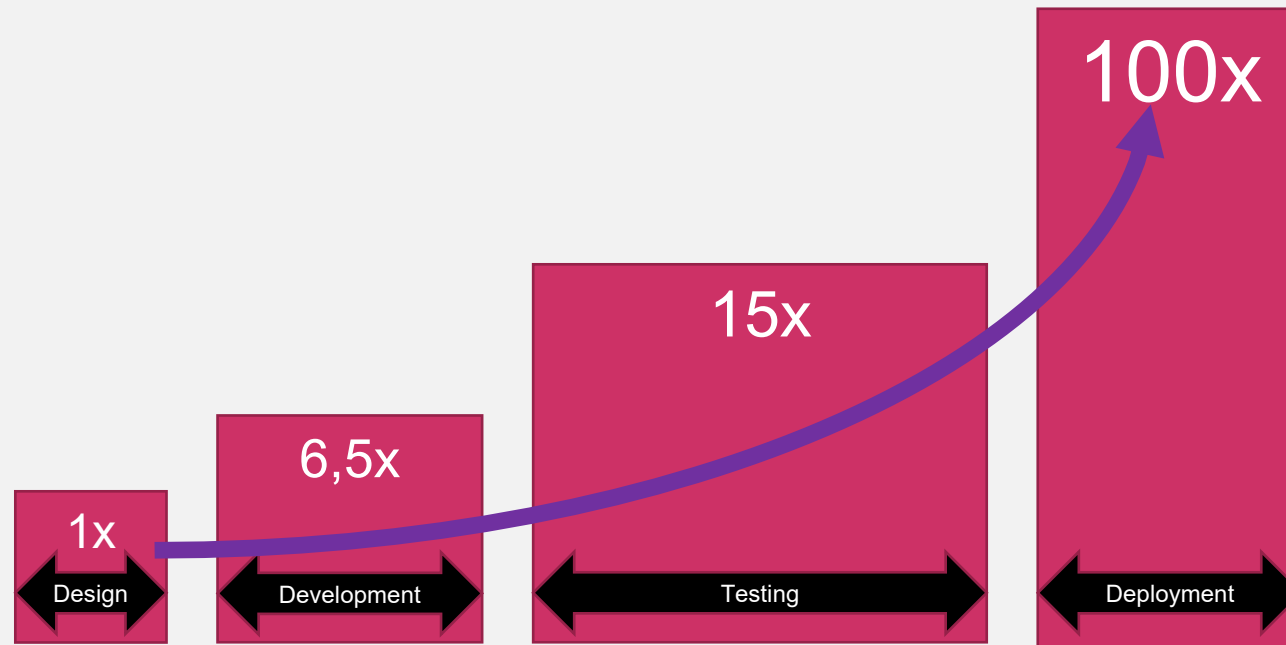
- Les projets agiles sont caractérisés par des modifications du code et des livraisons fréquentes. Chaque modification du code est susceptible d'introduire de la régression sur un contenu déjà testé, livré et validé par le client.
- C'est pourquoi il est important de tester non pas seulement les nouvelles fonctionnalités développées mais aussi veiller à ce que des défauts n'ont pas été introduits dans les fonctionnalités déjà livrées précédemment.
- Le périmètre augmente à chaque livraison de sprint et par conséquent les TNR deviennent de plus en plus consommateurs de ressources lorsqu'ils sont exécutés manuellement.
- La durée des campagnes de tests de non-régression risque de masquer l'effet positif du déploiement des méthodes Agiles. Il est donc impératif de raccourcir la durée des campagnes et d'en augmenter la fréquence grâce à l'automatisation des phases de test.



LE COÛT DE LA NON-QUALITÉ

LA QUALITÉ ET SCRUM

- Le coût de correction d'une anomalie augmente radicalement lors d'un passage d'une phase à l'autre.
- Il est donc indispensable de se donner les moyens de ne pas créer de dette technique,
- l'équipe prend le risque d'être noyée sous les correctifs sans possibilité de faire évoluer l'applicatif.

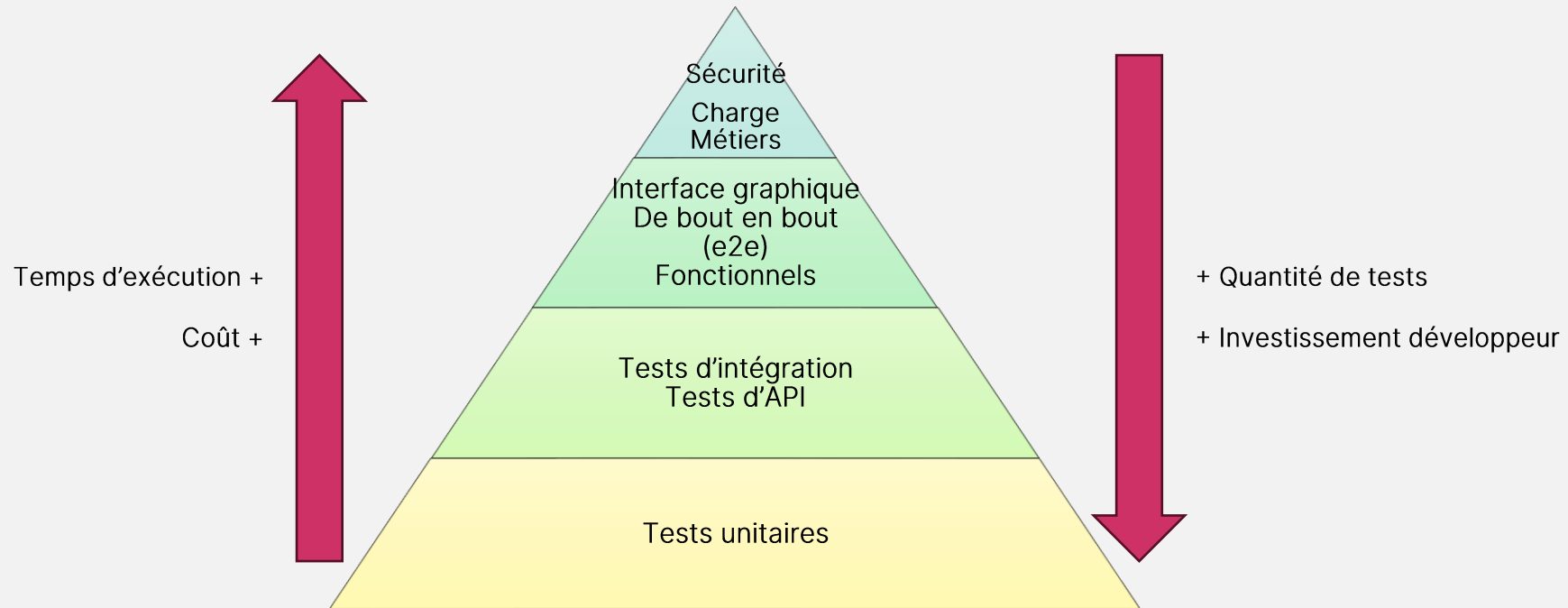




PYRAMIDE DES TESTS

LA QUALITÉ ET SCRUM

- Un outil pour se prémunir de la régression est la mise en place d'une stratégie de tests
- Il existe différents types de tests qui contribuent à renforcer la qualité



- Martin Fowler a défini le terme **Pyramide des tests**, expliquant que plus on monte dans la pyramide, plus on couvre d'éléments fonctionnels, mais plus ils sont lents et coûteux à exécuter.

TOUR D'HORIZON DES DIFFÉRENTS TYPES DE TESTS





LES TESTS UNITAIRES

LA QUALITÉ ET SCRUM

- **Ils consistent à tester individuellement les composants d'une application**
- **Ils valident que le code répond aux attentes en s'assurant :**
 - Qu'elles soient couvertes par le code écrit.
 - De la non-régression en cas de modification du code.
 - De la performance du code.
- **Le développeur est responsable des tests et du code qui composent le travail demandé**
- **Ils doivent être automatisés rapidement**
 - Ils assurent ainsi qu'aucune régression ne s'est produit sur le code testé.
 - Ils doivent être exécuté sur le poste de travail et sur la PIC

What is a unit test?

1. A test that isolates and verifies individual units of functionality
2. A way to ensure product code satisfies the user requirements
3. A test that ensures a specific method integrates properly with the code calling it
4. A technique for ensuring that units of co-dependent systems integrate correctly



LES TESTS D'INTÉGRATION

LA QUALITÉ ET SCRUM

- Ils valident la bonne interaction entre les différents modules dans leur environnement
- Ils peuvent mettre en évidence des problèmes d'interfaces entre différentes applications
 - Evolution du contrat d'interface
 - Impact d'une évolution dans un composant dépendant
- L'exécution de ces tests peut être réalisée en dehors de l'équipe de développement
- L'objectif est d'assurer que les différents composants logiciels interagissent entre eux conformément aux attentes.

What is integration test?

1. A test that runs during a Continuous Integration build
2. A test of multiple units of functionality
3. A test of a single unit of functionality
4. A test of the user interface



LES TESTS FONCTIONNELS

QUALITÉ ET SCRUM

- Ils ont pour objectif de vérifier la conformité de l'application.
- Ils se basent sur les spécifications fonctionnelles et techniques.
- L'automatisation de ces tests demande un effort important (temps et coûts).
- Ils peuvent être manuels (exploratoires) en utilisant un « cahier de recette »
 - Il faut qu'il soit rédigé au préalable.
 - Il regroupe les besoins client (Use Case).
- La personne en charge des tests l'utilise pour :
 - s'assurer du bon fonctionnement de l'application.
 - y consigner les résultats vis-à-vis des fonctionnalités attendues par le client.



TESTS DE CHARGE ET DE PERFORMANCE

LA QUALITÉ ET SCRUM

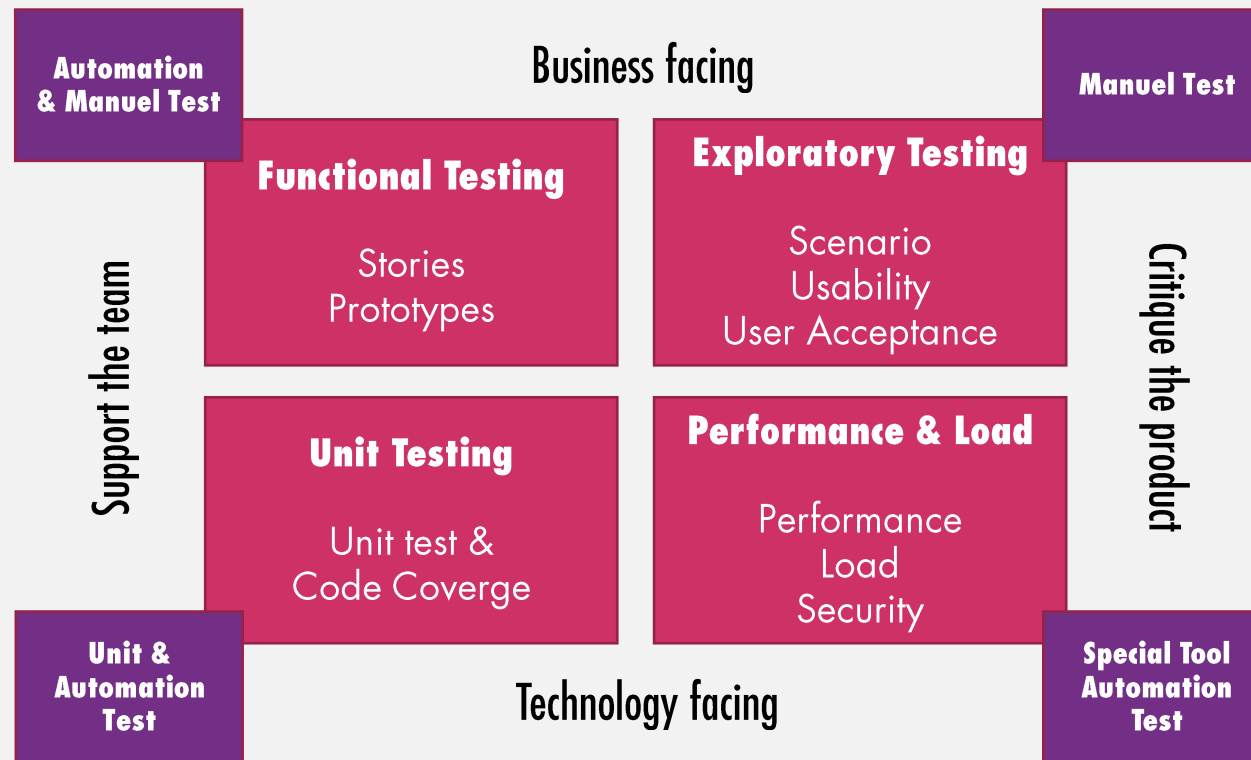
- **Ce sont des tests permettant de mesurer les temps de réponses du système en fonction des sollicitations.**
- **Les tests de charge simulent un nombre prédéfini d'utilisateurs en simultané pour :**
 - Mesurer le dimensionnement de l'infrastructure nécessaire (serveurs, bande passante sur le réseau, ...)
 - S'assurer du respect des exigences de performances dans le temps
- **Les tests de performance permettent de récupérer des métriques :**
 - Serveurs
 - Temps de réponses
 - Percentile CPU
 - RAM (employée, disponible,...)
 - Bande passante
 - Appels aux services externes
- **Le test de montée en charge permet d'évaluer sa capacité à supporter de plus en plus d'utilisateurs tout en maintenant une expérience utilisateur optimale et un fonctionnement correspondant au cahier des charges.**



QUADRANTS DE BRIAN MARICK

LA QUALITÉ ET SCRUM

- Ce diagramme montre les quatre quadrants qui reflètent les différentes raisons et objectifs des tests dans un projet agile
- Les quadrants de gauche incluent des tests qui aident les développeurs et qui représentent un nouveau concept : c'est la grande différence entre les tests sur un projet traditionnel et un projet agile



TEST DRIVEN DEVELOPMENT (TDD)





TEST DRIVEN DEVELOPMENT (TDD)

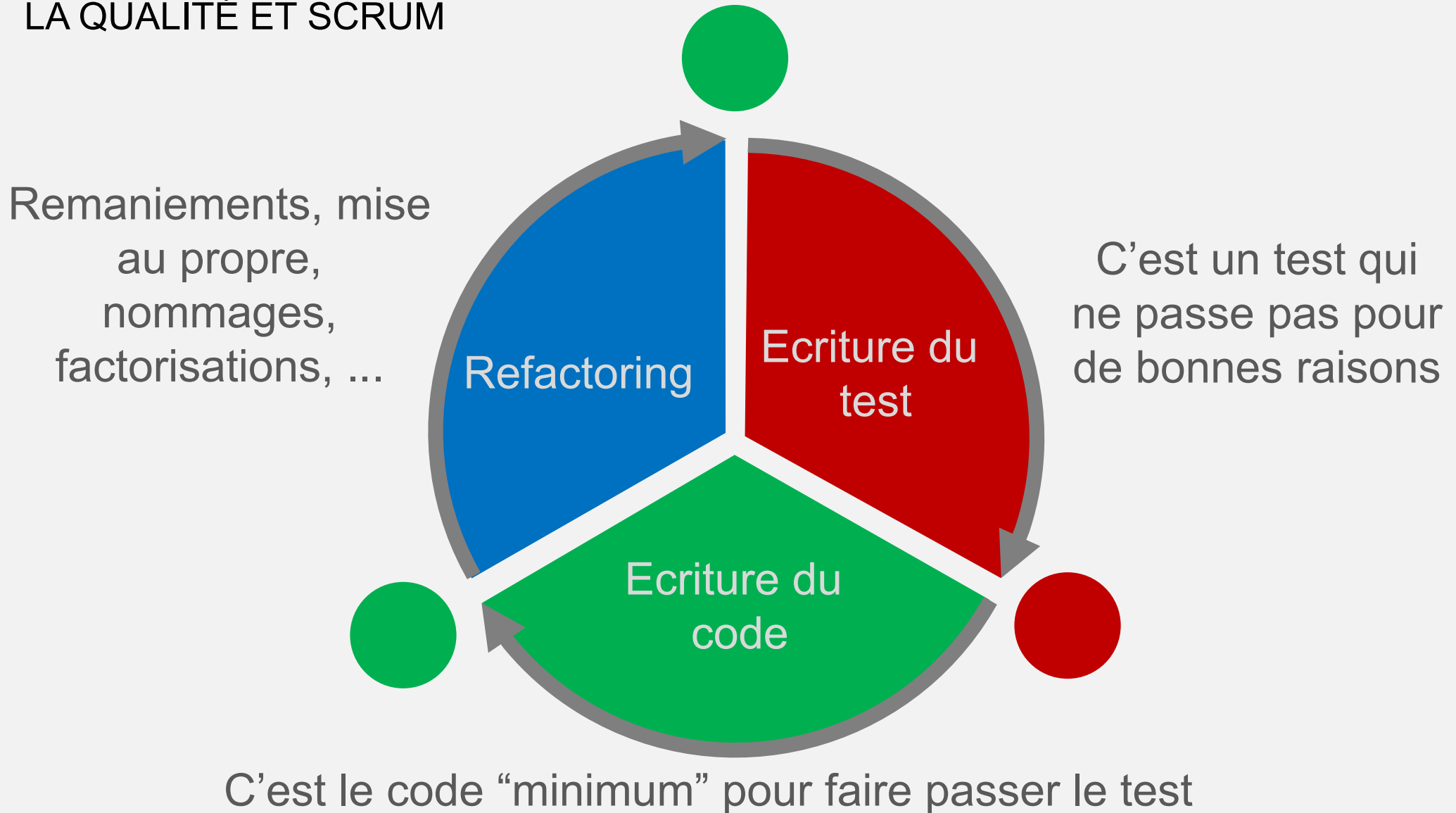
LA QUALITÉ ET SCRUM

- Il s'agit d'une technique de conception où le programmeur **écrit d'abord le test avant de produire le moindre code**.
 - Le développeur écrit ensuite le code pour que le test passe.
 - Une fois son test finalisé, il pourra être libre de refactorer autant qu'il le souhaite jusqu'à obtenir un code « propre ».
 - C'est une idée simple mais complexe à mettre en œuvre.
 - Courbe d'apprentissage plus lente de prime abord.
- Pourquoi?
 - Vérifier la bonne compréhension des fonctionnalités
 - Meilleure couverture de tests automatisés
 - Facilité d'écriture des tests avant le code « métier »
 - Ils servent à promouvoir et vérifier la qualité et la fiabilité du code



TDD – « RED / GREEN / REFACTOR »

LA QUALITÉ ET SCRUM



ACCEPTANCE TESTING DRIVEN DEVELOPMENT (ATDD)





ACCEPTANCE TESTING DRIVEN DEVELOPMENT (ATDD)

LA QUALITÉ ET SCRUM

➤ Les tests sont écrits en même temps que la description de la fonctionnalité

- Ils sont là pour valider l'adéquation du logiciel aux spécifications du client.
- Ils seront réalisés par le client pour valider la fonctionnalité.
- Ils renforcent la collaboration entre les équipes de dev et le client.

➤ Ils tiennent lieu de « contrat » entre l'équipe de développement et le métier.

- Le but est d'automatiser des tests (unitaire ou d'intégration) en fonction des critères formulés.
- Cela renforce la collaboration entre les équipes métiers et l'équipe de développement.
- On se concentre sur les résultats attendus :
 - passage d'un statut à un autre,
 - réponse d'une API,
 - temps de chargement d'une page,
 - etc.

- Permet aux différents acteurs du projet d'avoir une vision commune du résultat.
- Ils sont réalisés par le client final ou les utilisateurs : ils recettent le logiciel.

The practice of decomposing a requirement into failing tests is called :

1. Regression testing
2. Acceptance Test Driven Development
3. Object oriented requirements definition
4. Behavior Driven Development

BEHAVIOR- DRIVEN DEVELOPMENT (BDD)





BEHAVIOR-DRIVEN DEVELOPMENT (BDD)

LA QUALITÉ ET SCRUM

- **Il s'agit d'écrire des tests qui décrivent le comportement attendu du système et que tout le monde peut comprendre.**
- **Cette démarche permet de dérouler des scénarios fonctionnels complet depuis l'IHM pour simuler de utilisateurs.**
- **Exemple de question de la certification :**
 - Which answer best describes Behavior-Driven Development (BDD)?
 1. A development style that accounts for leadership style among team members.
 2. A technique for maintaining regression test harnesses.
 3. A way to organize unit tests based on class and method structures.
 4. A style of test-driven development focusing on user and system interactions.

AUTOMATISATION DES TESTS





AUTOMATISATION DES TESTS

LA QUALITÉ ET SCRUM

PSD1

> Il existe des outils pour automatiser les types de tests suivants :

- Tests unitaires.
- Tests d'intégrations
- Tests d'acceptation.
- Tests de comportement.
- Tests d'IHM.
- Tests d'accès aux données.
- Tests de performance.
- Smoke tests
- ... etc

Imp

> Les Frameworks de tests:

- Il existe différents langages de programmation que vous pouvez envisager pour réaliser des tests automatisés, tels que Java, Python, C#, etc.
- Un panel plus ou moins important de frameworks et d'outils est disponible pour réaliser ces tests (manuels ou automatisés).



AUTOMATISATION DES TESTS

LA QUALITÉ ET SCRUM

➤ LES FRAMEWORKS POUR LES TESTS UNITAIRES

- JUnit, TestNG (java)
- Jasmine, Karma, Mocha (javascript)
- NUnit (.Net)
- SimpleTest, PHPUnit, Atoum (PHP)
- dUnit (Delphi)
- cppUnit (C++) etc..





AUTOMATISATION DES TESTS

LA QUALITÉ ET SCRUM

➤ LES FRAMEWORKS POUR LE MOCKING

- Mockito, EasyMock, PowerMock (java)
- Sinon JS, Jest, Rhino Mocks (javascript)
- TypeMock, Moq (.Net)





AUTOMATISATION DES TESTS

LA QUALITÉ ET SCRUM

➤ LES OUTILS DE TESTS DE CHARGE

- JMeter, Gatling, Taurus, Locust



BONNES PRATIQUES DE L'ÉCRITURE DE TESTS UNITAIRES

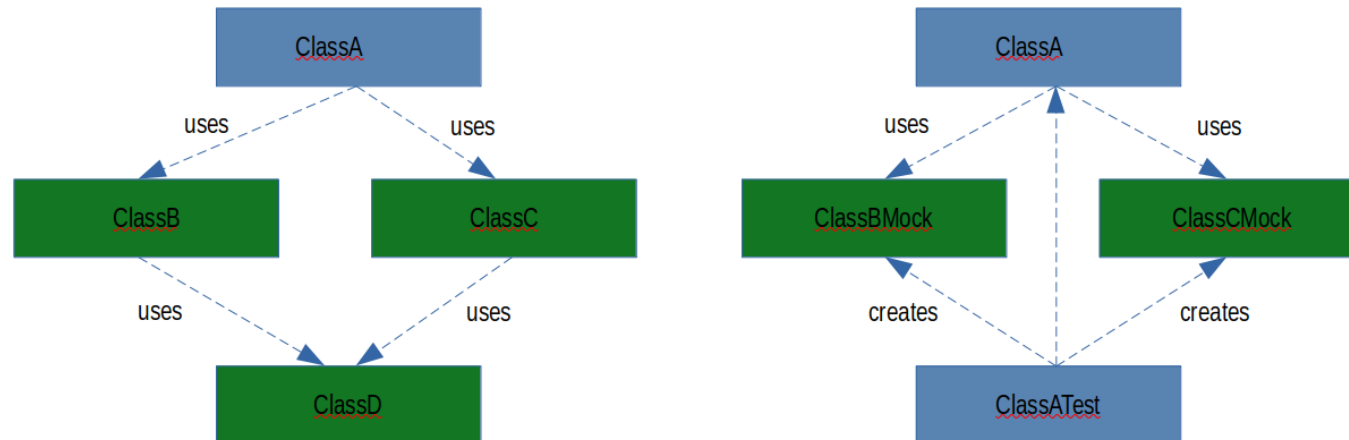




UTILISER UN MOCK

LA QUALITÉ ET SCRUM

- Les mocks (bouchons) sont des objets simulés qui reproduisent le comportement d'objets réels de manière contrôlée.
- Les frameworks de « mocking » permettent de :
 - bouchonner ou espionner des objets,
 - simuler et vérifier des comportements,
 - de simplifier l'écriture de tests unitaires (isolement).
- Seules les dépendances de premier niveau ont besoin d'être remplacées par des objets « mock » pour tester l'objet.





BONNES PRATIQUES DES TESTS UNITAIRES

LA QUALITÉ ET SCRUM

➤ **Stratégie de construction**

- Si vous n'avez pas encore de tests, ou pas le temps de coder une suite de tests complète, démarrez par des tests fonctionnels.
- Sur les parties critiques de votre application, renforcez vos tests fonctionnels avec des tests unitaires
- **Ne faites surtout pas de tests unitaires pour toutes vos fonctions**, lors d'un test unitaire vous testez toujours une fonctionnalité, mais sans qu'elle ne soit impactée par les autres modules en amont ou en aval.

➤ **Ne cherchez pas à atteindre un seuil de couverture de code sur l'intégralité de votre code.**

- Le but est de tester le code critique et de valeur, pas de faire de la couverture artificielle
- Il faut éviter de faire baisser le taux de couverture entre chaque release.

➤ **Ne pas effectuer des tests unitaires sur du code généré automatiquement ou sur du code identifié comme boilerplate (getters, setters)**



BONNES PRATIQUES DES TESTS UNITAIRES

LA QUALITÉ ET SCRUM

➤ Adopter la méthodologie « AAA » : Arrange, Act, Assert

- Dans la première étape, vous organisez les choses pour les tests : Arrange (contexte requis pour l'exécution du test et résultats attendus).
- Ensuite, vous agissez : Act. Ici, vous appelez la fonction testée et stockez ses résultats.
- Maintenant, vous affirmez si l'hypothèse est correcte : Assert. C'est l'essence d'un test unitaire, car cette partie teste enfin quelque chose. Le but est de vérifier si le résultat obtenu correspond à celui attendu.
- **La méthode « AAA » s'apparente à la méthodologie « BDD » (Behaviour Driven Development) avec Given-When-Then.**
- Bien aérer son test en ajoutant des commentaires pour distinguer les sections Given-When-Then.

➤ Écrire d'abord des tests avant de corriger les bugs

- Écrire le test qui permet de reproduire le bug.
- Vous laisserez un excellent test de régression pour repérer ce bogue à l'avenir. Et vous saurez que vous l'avez correctement corrigé lorsque le test qui a précédemment échoué commencera à réussir.



BONNES PRATIQUES DES TESTS UNITAIRES

LA QUALITÉ ET SCRUM

➤ Garder les tests courts

- Les fonctions courtes sont beaucoup plus faciles à lire et à comprendre.
- Puisque nous testons un morceau de logique à la fois, les tests ne devraient pas dépasser quelques lignes de code de toute façon.

➤ Rendre les tests simples

- Évitez la logique complexe dans les tests.

➤ Comme tout autre code, les tests sont également soumis à une refactorisation. Il faudra trouver un bon équilibre entre simplicité et répétitivité.

➤ Rendre les tests performants

- Les tests unitaires doivent pouvoir s'exécuter sur toutes les machines. Votre équipe devrait les exécuter plusieurs fois par jour.
- Ils s'exécuteraient à la fois pendant les builds locaux et dans votre Intégration Continue.
- S'assurer de « mocker » toutes les dépendances externes qui pourraient le ralentir, comme les appels d'API, les bases de données ou l'accès au système de fichiers.



BONNES PRATIQUES DES TESTS UNITAIRES

LA QUALITÉ ET SCRUM

➤ Séparez les tests de l'implémentation

- Nommez les classes de test de la même manière que les classes qu'elles testent (avec le suffixe Test)

➤ Utiliser des noms de méthodes de test explicites

- La première chose que vous voyez lorsque le test échoue est son nom (correspondant au nom de la méthode).
- Ne pas avoir peur d'un nom de méthode à rallonge (par exemple `it('should return 0 for an empty cart')` est bien meilleur que `it('works for 0')` ou `it('empty cart')`). Le nom de la méthode devrait fournir suffisamment d'informations pour comprendre ce que fait le test.
- Utiliser des noms explicites normalisés comme par exemple `calculateTotalShouldReturnZeroWhenCartIsEmpty()`



BONNES PRATIQUES DES TESTS UNITAIRES

LA QUALITÉ ET SCRUM

➤ Garder les tests « Stateless » (sans état)

- Les tests ne doivent rien changer en dehors de leur portée ou laisser des effets de bord.
- Les tests doivent être indépendants les uns des autres.
- Si vous avez besoin d'un arrangement répétitif complexe, utilisez les mécanismes de configuration (setup ou teardown) fournis par les frameworks.

➤ Écrire des tests déterministes

- Si le test réussit - il devrait toujours réussir, et s'il échoue - il devrait toujours échouer.
- L'heure de la journée, la disposition des étoiles ou le niveau de la marée ne devraient pas affecter cela.

➤ Tester une exigence à la fois

- Utiliser une assertion par test et privilégier les assertions spécifiques :
(Assert.assertEquals("AA", "BB"))
est plus précis que
(Assert.isTrue("AA".equals("BB")))

EXEMPLES DE QUESTIONS DE LA CERTIFICATION





EXEMPLES DE QUESTIONS DE LA CERTIFICATION:

LA QUALITÉ ET SCRUM

- What is the primary purpose of refactoring?
 1. Making sure that the code is readable and maintainable.
 2. Removing all bugs that were found during regression tests.
 3. Creating better technical documentation.
 4. Ensuring that all factors are constantly aligned.
- What would NOT be considered refactoring?
 1. Changing external interface or API
 2. Reordering method parameters to improve readability
 3. Extracting methods
 4. Renaming things to be more logical
 5. Extracting interfaces
- The right time to refactor code using Test Driven Development is:
 1. After the test passes
 2. After the test fails
 3. When I can see the "debt" building in the code
 4. When preparing legacy code for unit tests



EXEMPLES DE QUESTIONS DE LA CERTIFICATION

LA QUALITÉ & SCRUM

- Which is a benefit of refactoring code ?
 1. To keep the code moving.
 2. To have all code in a single file for easy printing and code reviews.
 3. Improves the external contract representing the system under test.
 4. To improve readability and maintainability.
- You are asked to refactor part of the codebase for Application X. When you are done, all unit tests pass with 50% code coverage. What can you infer from this ?
 1. At least 50% of Application X functions correctly.
 2. You did not break any existing unit tests.
 3. Less than 50% of Application X functions correctly.
 4. There are no bugs present in Application X.
- What is Technical Debt ?
 1. The average time or money a Scrum Team spends per Sprint on bug fixes.
 2. A term representing the eventual consequences of poor technical choices.
 3. Code that has not been commented or documented.
 4. The money an organization owes to tool and hardware vendors.



EXEMPLES DE QUESTIONS DE LA CERTIFICATION

LA QUALITÉ & SCRUM

- Which three describe Test-Driven Development? (Choose 3 answers)
 1. A predictable way to develop working , clean code.
 2. An incremental and emergent approach to software design.
 3. Creating a manual test script before writing code.
 4. A software development technique based on automated tests.
- What does a test written with Test Driven Development represent?
 1. Something that completes the test coverage of a system.
 2. A technical requirement that must be satisfied.
 3. A bug that will be uncovered.
 4. An assignment from the lead quality engineer.



EXEMPLES DE QUESTIONS DE LA CERTIFICATION

LA QUALITÉ & SCRUM

- The right time to refactor code using Test Driven Development is:
 1. After the test passes
 2. After the test fails
 3. When I can see the "debt" building in the code
 4. When preparing legacy code for unit tests
- Why does a test written using TDD (Test Driven Development) always fail?
 1. Because the test has not been refactored
 2. Because the tests are checked in before the product code exists.
 3. Because it has to be put into an automated test harness to be run.
 4. Because the product code to satisfy the test does not yet exist.



EXEMPLES DE QUESTIONS DE LA CERTIFICATION

LA QUALITÉ & SCRUM

- While practicing Test Driven Development, what is done after the test fails?
 1. Write the product code to satisfy the test.
 2. Run it again to make sure it really fails.
 3. Refactor the test so the code passes.
 4. Meet with the business analyst to ensure that the test is correct.
- Which four are benefits of Test Driven Development? (Choose 4 answers)
 1. It causes you to construct a test harness that can be automated.
 2. It reduces the cost of maintenance over time.
 3. It promotes good design and separation of concerns.
 4. It ensures no defects are present in the code.
 5. It improves quality and reduces bugs.

PAIR- PROGRAMMING





QU'EST CE QUE LE PAIR PROGRAMING ?

PAIR-PROGRAMMING

➤ Le pair-programing

- c'est 2 personnes assises l'une à côté de l'autre, travaillant sur la même fonctionnalité et le même ordinateur

➤ Pourquoi ? Quels avantages ?

- Renforcement des bonnes pratiques de développement
- La fonctionnalité est développée plus vite
- La fonctionnalité est de meilleure qualité
- Harmonisation des pratiques de développement
- Former les développeurs plus juniors

➤ Qui fait quoi ?

- Le « driver » a les mains sur le clavier
 - Se concentre sur ce que le code doit faire, et de le faire bien
- Le navigateur est assis à côté : Ne s'occupe pas de « ; » manquants mais ...
 - Est-ce que ce code s'inscrit bien dans l'architecture actuelle ?
 - Est-ce que ce code est dupliqué ? Devrait-on le factoriser ?
 - Le code est-il clair ?
 - La conception est-elle bonne ?



PAIR-PROGRAMMING : QUESTIONS

EXEMPLES DE QUESTIONS DE LA CERTIFICATION

➤ WHY MIGHT A DEVELOPMENT TEAM CHOOSE TO DO PAIR PROGRAMMING?

- Information sharing and learning
- It can be a fun way to work
- Improving code quality
- Efficiency
- All of the above

➤ WHAT IS PAIR PROGRAMMING?

- Managers doing performance reviews by comparing one programmer's code to another's.
- Two programmers write code separately. A third programmer integrates the work.
- The Scrum Team is divided into several two-person Development Teams that consolidate work in a Scrum of Scrums.
- A developer and a tester work together to write and test code.
- Two developers writing code together, providing constant peer review.

CLEAN CODE





LES FONDAMENTAUX

CLEAN CODE

- **Bjarne Stroustrup pense qu'un Clean Code est tout simplement un code élégant et efficient. un code élégant est un code qui fait plaisir à ses lecteurs et qui est écrit d'une manière qui empêcherait les bugs de s'y cacher ...**
 - Créateur du C++
- **Grady Booch pense qu'un Clean Code est un code simple, direct et n'obstrue en aucun cas le design de l'application.**
 - Co-créateur du langage UML et concepteur de la méthode Booch
- **Ward Cunningham pense qu'un Clean Code est un code qui ravit son lecteur, mais peut également se transformer en « Beautiful Code » quand il est écrit d'une manière faisant croire à son lecteur que le langage de programmation employé aurait été destiné à résoudre le besoin en question**
 - Co-créateur de XP et signataire du Manifeste Agile
- **Michael C. Feathers pense qu'un Clean Code est un code écrit par un développeur attentionné et n'a nullement besoin d'optimisation.**
 - L'un des principaux prédicateurs de l'approche TDD et des bonnes pratiques orientées objet,
- **Dave A. Thomas pense qu'un Clean Code est un code pouvant changer de mains sans la moindre difficulté, un code testable et testé, explicite et exprime les choses d'une seule manière.**
 - Fondateur de Eclipse et de VisualAge
- **Robert C. Martin pense qu'un Clean Code est un code dont a pris soin.**
 - Aussi appelé Uncle Bob, un des principaux prédicateurs de l'approche TDD et signataire du Manifeste Agile,

EN SYNTHÈSE, UN « CLEAN CODE » EST UN CODE ÉLÉGANT, CLAIR, FAIT AVEC ATTENTION PAR DES GENS PASSIONNÉS, TESTABLE, DÉCOUPLÉ ET SURTOUT EXPRESSIF

Source : <http://blog.cellenza.com/archi-patterns-bp/127609/>



LES PRINCIPES

CLEAN CODE

➤ DRY : DON'T REPEAT YOURSELF

- Ne pas dupliquer de code (dans un context d'exécution)

➤ SOLID (SINGLE RESPONSIBILITY, OPEN-CLOSED, LISKOV SUBSTITUTION, INTERFACE SEGREGATION AND DEPENDENCY INVERSION)

- Les 5 principes de base du développement orienté objet.

➤ LIM : LESS IS MORE

- Factorisation et limitation du code

➤ YAGNI : YOU AREN'T GONNA NEED IT

- Ne pas ajouter de fonctionnalité à un logiciel tant que celle-ci n'est pas absolument nécessaire

➤ KISS : KEEP IT SIMPLE, STUPID (OU KEEP IT STUPID, SIMPLE 😊)

➤ SOC : SEPARATION OF CONCERNS

➤ DEMETER : DON'T TALK TO STRANGERS

- Un objet doit éviter d'invoquer des méthodes d'un membre objet retourné par une autre méthode



ATELIER LEGO 1

CLEAN CODE

- AVEC LE NOMBRE DE PIÈCES QUE VOUS VOULEZ, REPRÉSENTEZ UN ANIMAL EN 2 MINUTES.
- FAITES UNE CAGE POUR VOTRE ANIMAL EN 2'30 MINUTES.
- QU'AVONS-NOUS APPRIS ?
 - Quand nous créons un projet, nous avons un programme parfait
 - A chaque ajout de code nous nous éloignons de cet idéal
 - Garder un problème aussi simple et petit possible
 - L'homme adore remplir le temps, vous auriez eu 30" vous auriez fait plus simple.
 - Si je vous donne 30' (ou un peu plus) :





QUID DE LA VISION GLOBALE ?

CLEAN CODE

- **dans un contexte agile, l'équipe de développement s'engage**
 - sur une estimation et sur une conception technique en début de Sprint.
 - à produire un code de qualité via les concepts de « Clean Code ».
- **En revanche, le principe de faire une conception par Sprint augmente le risque de perdre la vision globale de l'application. Cela va impliquer une complexité croissante du code et de sa maintenance.**

LA DETTE TECHNIQUE





ATELIER LEGO 2, CONSTRUIRE UNE MAISON

LA DETTE TECHNIQUE

**VOUS N'AVEZ PAS LE DROIT DE TOUCHER À L'INCRÉMENT PRÉCÉDENT
(REFACTORING INTERDIT) SPRINT DE 2 MINUTES**

- **SPRINT 1 : CONSTRUIRE 2 MURS CONNECTÉS.**
- **SPRINT 2 : FAIRE UNE FENÊTRE SUR UN MUR.**
- **SPRINT 3 : ENTièrement FERMÉE.**
- **SPRINT 4 : AJOUTER UN TOIT EN PENTE.**
- **SPRINT 5 : AJOUTER UNE CHEMINÉE.**
- **SPRINT 6 : AJOUTER UNE PORTE AU RDC.**
- **SPRINT 7 : AJOUTER UNE FENÊTRE SUR UN SECOND MUR.**
- **SPRINT 8 : LES TOITS EN PENTE NE SE VENDENT PLUS : UN TOIT PLAT !**



QUÉSACO ?

LA DETTE TECHNIQUE

➤ Elle intervient :

- en cas de non-respect des règles et conventions
- en l'absence d'optimisation
- quand on développe sans prendre de la hauteur (vision sprint versus vision produit) :
 - Manque de mutualisation de code.
 - Code non générique

➤ Elle induit une augmentation du coût final du produit. Comme une dette financière.

- plus on retarde son règlement, plus les intérêts à payer sont importants.

➤ Si on attend trop, on finit par se retrouver avec un produit pour lequel tous les coûts sont absorbés par la maintenance.

- Il n'évolue plus.



SON ORIGINE

LA DETTE TECHNIQUE

EXCESSIVE ET IMPRUDENTE

MODEREE ET PRUDENTE

VOLONTAIRE

Dettes produites par des équipes qui décident délibérément de faire du "quick and dirty" parce qu'elles ne peuvent prendre le temps nécessaire pour concevoir leur code proprement.

Ex: site d'e-commerce qui réalise une grande partie de son CA pendant la période de Noël. Les fonctionnalités doivent sortir avant cette période.

Une dette complètement délibérée pour livrer à temps une version du produit ne vaut peut-être pas la peine d'être remboursée si les intérêts sont suffisamment petits.

Ex: une partie de code rarement modifiée.

INVOLONTAIRE

Dettes souvent dues à des équipes ignorant les pratiques de conception et d'architecture les plus basiques.

Ex: startups pour lesquelles il est plus important de valider une idée ou d'être le premier à se lancer sur un marché que d'écrire du code de qualité.

Dettes techniques existant dans les équipes mêmes les plus expérimentées car inévitable. Souvent ce n'est qu'à la fin d'un projet que l'on se rend réellement compte de ce que la conception aurait dû être, même si le code et l'architecture mis en place sont très bons et très bien pensés.



DE QUOI EST-ELLE COMPOSÉE?

LA DETTE TECHNIQUE

- La qualité et la teneur des spécifications (leur clarté, le « juste assez »)
- Le nombre de tests et leur utilité (la couverture, leur pertinence)
- Les conventions de code (la maintenabilité, le respect des standards)
- La bonne utilisation des concepts de design
- La mise en place d'un cadre d'intégration et de remontée de *feedbacks* (automatisation...)
- La documentation technique, fonctionnelle et utilisateur
- ...



LES SIGNAUX PERCEPTIBLES ET IMPACTS MESURABLES

LA DETTE TECHNIQUE

- **L'insatisfaction des utilisateurs lors de la revue ou de la livraison (non-acceptation)**
- **Un ratio de plus en plus important entre la durée de la livraison et la complexité d'un item**
- **Le nombre de bugs et d'incidents relevés sur une période donnée**
- **La durée des investigations sur des produits, composants ou code hérités**
- **La dépendance de l'équipe des rôles spécialistes**
- **Le nombre d'heures supplémentaires que travaille l'équipe de développement subissant de la pression**
- **Le score que peut donner l'équipe de développement quant au plaisir qu'elle a à travailler sur le projet**
- **Le besoin de formation et d'outils**
- **...**



ATELIER LEGO 2, CONSTRUIRE UNE MAISON

LA DETTE TECHNIQUE

VOUS AVEZ LE DROIT AU REFACTORING ! SPRINT DE 2 MINUTES

- **SPRINT 1 : CONSTRUIRE 2 MURS CONNECTÉS.**
- **SPRINT 2 : ENTIÈREMENT FERMÉE.**
- **SPRINT 3 : FAIRE UNE FENÊTRE SUR UN MUR.**
- **SPRINT 4 : AJOUTER UN TOIT PLAT.**
- **SPRINT 5 : AJOUTER UNE CHEMINÉE.**
- **SPRINT 6 : AJOUTER UNE PORTE AU RDC.**
- **SPRINT 7 : LES TOITS PLATS NE SE VENDENT PLUS : UN TOIT EN PENTE !**
- **SPRINT 8 : AJOUTER UNE FENÊTRE SUR UN SECOND MUR.**

QUALIMÉTRIE





COMMENT GÉRER LA DETTE TECHNIQUE

QUALIMÉTRIE

- **Objectif : limiter bugs, performance, faciliter les évolutions du code**
- **Actions à mener**
 - Faire un suivi des dettes techniques
 - Planifier la dette dans les sprints
 - Mettre en place des tests
 - Réécrire le code si nécessaire
- **Si nouveau projet : Mise en œuvre dès le début**
- **SI projet existant : Mise en place a minima d'une estimation et du suivi de son évolution**
- **Comment rembourser la dette technique ?**
 - Eviter d'alourdir la dette sur les nouvelles fonctionnalités développées
 - Assainir les unités de code impactées (fichiers/classes) par les développements réalisés
 - Allouer du temps lors des itérations au rattrapage de la dette
 - Introduire dans « Definition Of Ready » : Définir les exigences de qualimétrie pour la « User Story »
 - Intégrer les métriques et seuils de qualité dans la « Definition Of Done »



DES MÉTRIQUES DE LA QUALITÉ LOGICIEL

QUALIMÉTRIE

- **Profondeur de l'arbre d'héritage (depth of inheritance)**
 - Nombre de niveau d'héritage d'une classe
- **LA COMPLEXITÉ CYCLOMATIQUE D'UNE MÉTHODE (Cyclomatic complexity)**
 - **CYC = nombre de noeuds if, case, while...**
 - Plus une classe a une CYC élevée, moins elle sera simple à lire et à comprendre.
- **L'INDICE DE SPÉCIALISATION D'UNE CLASSE**
 - **$SPE = (S \cdot P) / Nb$**
 - S = Nombre de méthodes surchargées
 - P = Profondeur d'héritage depuis Object
 - Nb = nombre de méthodes de la classe
 - Une classe dont le SPE est trop élevé devra être refactorée.
- **Duplication de code**
- **Nombre de méthodes/classes trop longues**
- **Nombre de classes par librairie**
- **Nombre de méthodes par classe**



DES MÉTRIQUES DE LA QUALITÉ LOGICIEL

QUALIMÉTRIE

➤ Couverture de code (Code Coverage)

- C'est une mesure utilisée pour décrire le taux de code source exécuté d'un programme quand une suite de test est lancée
- Attention, le taux de couverture ne montre pas la pertinence du code testé

➤ Plusieurs autres métriques (ou « critères ») existent et permettent de calculer la couverture de code à différents niveaux de granularité :

- Method coverage : Est-ce que toutes les méthodes ont été exécutées ?
- Branch coverage : Est-ce que les structures de contrôle (if ou while par exemple) ont été testées pour les 2 cas vrai et faux ?
- Path coverage : Est-ce que tous les chemins possibles d'exécution des fonctions d'un programme ont été exécutés ?

EXEMPLES DE QUESTIONS DE LA CERTIFICATION





LA QUALITÉ & SCRUM : QUESTIONS

EXEMPLES DE QUESTIONS DE LA CERTIFICATION

- **Cyclomatic complexity is a metric for :**
 1. Measuring branching structures and nesting levels of code
 2. Determining the number of unit tests required to ensure correctness
 3. Demonstrating code is well structured and cleanly implemented
 4. Assessing if code is written in as few lines as possible
- **Which three of the following are code quality metrics? (Choose 3 answers)**
 1. Cycle time per Product Backlog item.
 2. Cyclomatic complexity
 3. Class coupling
 4. Unit tests per class
 5. Number of check-ins per day
 6. Depth of inheritance
- **Which are two benefits of establishing naming conventions for code? (Choose 2 answers)**
 1. To reduce friction in a shared codebase.
 2. To ensure that orphan are not created.
 3. To communicate the identity of the developer who worked on the code.
 4. To make the code more readable.
 5. To make it easy to distinguish between different software products.



LA QUALITÉ & SCRUM : QUESTIONS

EXEMPLES DE QUESTIONS DE LA CERTIFICATION

- **What is code coverage ?**
 1. The number of developers that understand how the code works.
 2. The percent of code in version control included in a build.
 3. The degree to which the system under test has been exercised by tests.
 4. An insurance policy that covers the code for defects.
- **What does Code Coverage show?**
 1. The quality of unit tests being written.
 2. When a feature is done.
 3. The ratio of the number of tests to lines of code in the system being tested.
 4. Code being exercised by tests.
 5. The absence of defects in code.
- **Which is the most reliable form of technical documentation ?**
 1. Release notes
 2. A UML model
 3. A help file
 4. A spreadsheet of passing manual tests
 5. The Development Team's whiteboard
 6. A passing test harness with clear naming and vocabulary



LA QUALITÉ & SCRUM : QUESTIONS

EXEMPLES DE QUESTIONS DE LA CERTIFICATION

- **Which model or idea increases code testability?**
 1. CI (Continuous Integration)
 2. SOLID (Single responsibility, Open-closed. Liskov substitution, Interface segregation and Dependency Inversion)
 3. PLP(Principle of Least Privilege)
 4. INVEST (Independent, Negotiable. Valuable, Estimable. Small Enough, Testable)
- **What are some disadvantages of code coverage as a measurement for how well a system or product is tested? (Choose 3 answers)**
 1. It does not measure usability.
 2. It is too complicated to explain to management.
 3. It only provides insights to programmers.
 4. It could create incentives to write tests that simply increase code coverage, rather than tests that find bugs without increasing coverage.
 5. It does not ensure that the most important or highest risk areas of the code are being exercised by tests.

SONARQUBE





SONARQUBE

UN OUTIL DE VÉRIFICATION AUTOMATIQUE DU CODE - *QUALIMÉTRIE*

➤ Il s'intègre :

- Au workflow de développement pour détecter rapidement les problèmes dans le code.
- Aux plateformes d'intégration continue pour garantir un code répondant aux normes définies.

➤ Il aide

- Grâce à ses indicateurs, à produire un code propre.
- Comme support aux revues de code.

➤ Il s'applique à tout type de code :

- code source, code de test, Infrastructure as Code, scripts, etc.

➤ L'approche "Clean as You Code" de Sonar :

- Vise à limiter le nombre d'erreurs pouvant être découverte lors des revues de code à un stade trop avancé du développement.
- Les Quality Gate alertent et informent sur le code (existant, ajouté ou modifié) qui ne respectent pas les Ruleset.

➤ Objectif :

- Maintenir des normes élevées et se concentrer sur la qualité du code.



SONARQUBE

LES DIFFÉRENTS TYPES D'INDICATEURS – *LES QUALITY GATE*

➤ **Sonar propose tout un ensemble d'indicateurs**

- Leur le seuil est ajustable au sein d'une Quality Gate.
- Il est recommandé de les spécialiser pour chaque projet et techno.
- Différentes stratégies peuvent être mises en place.
- Toute évolution doit être partagée avec les intervenants du projet → les impacts peuvent être énormes.

➤ **Suivi des différentes métriques**

- Le plus important n'est pas la valeur à date mais la **tendance**
- Fichiers (nb lignes,...), violations, duplications...
- Consensus à trouver sur les valeurs à suivre et les seuils

➤ **Les tableaux de bord standard ne présentent que certains indicateurs**

- D'autre sont disponibles via les API. Par exemple, Grafana peut les transformer pour consolidation

➤ **SonarQube et ses indicateurs ne sont JAMAIS un but en soi.**



SONARQUBE

LES ISSUES – TYPES ET SÉVÉRITÉS

- **Lors d'une analyse, pour chaque morceau de code qui enfreint une règle une issue est créée.**
- **Pour chaque langage et projet, Un ensemble de règles (RuleSet) est associé à un profil de qualité (Quality Profile).**
- **Il existe trois types d'issues :**
 - **BUG** : Une portion de code pouvant entraîner une erreur ou un comportement inattendu au moment de l'exécution.
 - **VULNERABILITE** : Une portion de code ouverte à une attaque : crypto faible, accès chemins locaux, identifiants, injection SQL , ...
 - **CODE SMELL** : Un problème de maintenabilité qui rend votre code peu lisible/compréhensible et difficile à maintenir.
- **Chaque issue a une sévérité :**
 - ❗ • **BLOCKER** : forte probabilité d'avoir un impact sur le comportement de l'application → doit être corrigé sans attendre.
 - ⬆️ • **CRITICAL** :
 - Un bug avec une faible probabilité d'impacter le comportement de l'application.
 - Un problème qui représente une faille de sécurité.
 - ⬆️ • **MAJOR** : défaut de qualité qui peut avoir un impact important sur la productivité du développeur.
 - ⬇️ • **MINOR** : défaut de qualité qui peut avoir un léger impact sur la productivité du développeur
 - ℹ️ • **INFO** : ni un bug ni un défaut de qualité, juste un constat.



SONARQUBE

EXEMPLES D'ÉCRANS

OverviewIssuesSecurity HotspotsCodeMeasuresActivity

Project SettingsProject Information

master7.7k Lines of Code - Version 2.0.0-SNAPSHOTSet as homepageTake the Tour

Quality Gate

Passed

The last analysis has warnings. See details

Last analysis 3 months ago

New CodeOverall Code

Vulnerabilities

4 Open issues

Accepted issues

0

Valid issues that were not fixed

Security Hotspots

6

Bugs

30 Open issues

Coverage

0.0%

On 2.7k lines to cover.

Code Smells

188 Open issues

Duplications

8.6%

On 11k lines.

FiltersClear All Filters

Language

Search for languages...

Java168

Python Notebooks85

Python85

JavaScript83

C#80

5 shown Show More

Add to selectionCtrl + click

Type

Bug168

Vulnerability32

Code Smell456

Security Hotspot39

Add to selectionCtrl + click

Severity

Blocker14

Critical28

Major103

Minor23

Info0

Tags

Repository

Status

Available Since

Security Category

Search for rules...

Select rules

Navigate to rule

168 rules

"equals()" should not be used to test the values of "Atomic" classes

BugMedium

Java - multi-threading

"@Autowired" should only be used on a single constructor

BugHigh

Java - spring

"@Bean" methods for Singleton should not be invoked in "@Configuration" when proxyBeanMethods is false

BugMedium

Java - spring

"@Controller" classes that use "@SessionAttributes" must call "setComplete" on their "SessionStatus" objects

BugBlocker

Java - spring

"@NonNull" values should not be set to null

BugLow

Java - symbolic-execution cwe

"@PathVariable" annotation should be present if a path variable is used

BugMedium

Java - spring

"@Qualifier" should not be used on "@Bean" methods

BugMedium

Java - spring

"@Scheduled" annotation should only be applied to no-arg methods

BugHigh

Java - spring

"@SpringBootApplication" and "@ComponentScan" should not be used in the default package

BugBlocker

Java - spring

"BigDecimal(double)" should not be used

BugMedium

Java - cart

"Collection.toArray()" should be passed an array of the proper type

BugLow

Java

"compareTo" results should not be checked for specific values

C1-INTERNAL

183

SOURCE CONTROL & CI/CD





CONTRÔLE DE SOURCE

SOURCE CONTROL & CI/CD

- **Définition:** Le contrôle de source, également connu sous le nom de gestion de version, est une pratique de gestion des changements appliquée aux documents, programmes, et autres informations stockées comme fichiers informatiques.
- **Objectif:** Permettre aux équipes de suivre et de gérer les modifications apportées au code source au fil du temps, facilitant ainsi la collaboration et la coordination entre les développeurs.
- **Importance dans Agile et Scrum:** Dans les méthodologies Agile et Scrum, le contrôle de source est essentiel pour maintenir l'organisation et la transparence des modifications, contribuant à l'adaptabilité et à l'efficacité du processus de développement.

« Un logiciel de gestion de versions (ou VCS en anglais, pour version control system) est un logiciel qui permet de stocker un ensemble de fichiers en conservant la chronologie de toutes les modifications qui ont été effectuées dessus. Il permet notamment de retrouver les différentes versions d'un lot de fichiers connexes. »



CONTRÔLE DE SOURCE

SOURCE CONTROL & CI/CD

➤ Avantages:

- **Collaboration Améliorée:** Permet à plusieurs membres de l'équipe de travailler simultanément sur différents aspects d'un projet sans interférence.
- **Historique Complet:** Chaque modification est enregistrée avec des détails sur l'auteur, la date, et la raison du changement, ce qui est crucial pour la révision et l'audit.
- **Gestion des Conflits:** Offre des outils pour fusionner les changements et résoudre les conflits de manière efficace.

➤ Pratiques Clés dans Scrum:

- **Intégration Continue:** Les développeurs fusionnent régulièrement leurs modifications dans une branche partagée, réduisant ainsi les conflits et les régressions.
- **Revue de Code:** Les Pull Requests ou Merge Requests facilitent les revues de code, améliorant ainsi la qualité et la conformité.
- **Branching Stratégique:** Utilisation de branches pour les fonctionnalités, les corrections de bugs, ou les releases, permettant une meilleure organisation et gestion des versions.



DÉFINITION DE L'INTÉGRATION CONTINUE

SOURCE CONTROL & CI/CD

➤ Définition:

- L'Intégration Continue (CI) est une méthode de développement logiciel dans laquelle les modifications de code sont régulièrement et automatiquement testées et intégrées dans un dépôt central.
- Cette pratique encourage les développeurs à partager leurs travaux fréquemment, idéalement plusieurs fois par jour, permettant ainsi une détection et une résolution précoces des problèmes, et favorisant une collaboration plus efficace au sein des équipes de développement.

➤ Objectif:

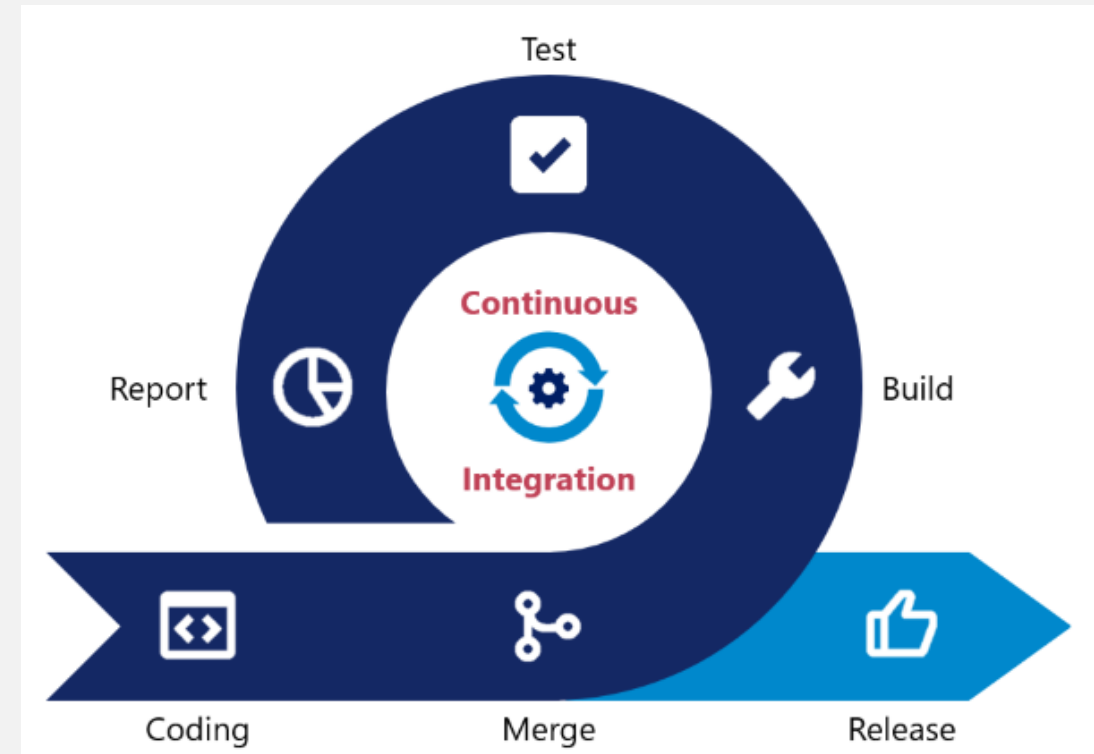
- Le principal objectif de CI est de détecter et de résoudre les conflits de codage tôt et régulièrement pour éviter des problèmes majeurs.

➤ Processus:

- Chaque merge request est automatiquement testée et vérifiée, ce qui permet d'identifier rapidement les erreurs et les incompatibilités.

➤ Avantages de l'Intégration Continue :

- **Réduction des conflits de code:** CI minimise les problèmes liés au merge de code en intégrant régulièrement les changements.
- **Détection rapide des erreurs:** Les tests automatiques dans CI aident à identifier et à résoudre les bugs rapidement.
- **Livraison accélérée:** CI permet une mise en production plus rapide en assurant une base de code toujours prête pour la livraison.





DÉFINITION DU CONTINUOUS DELIVERY

Vérifier dans
les questions

SOURCE CONTROL & CI/CD

- **Définition** : Le Continuous Delivery est une pratique de développement logiciel où les modifications de code sont automatiquement testées et préparées pour une livraison en production.
- **Objectif** : Assurer qu'une version du logiciel soit toujours en état de être déployée à tout moment.
- **Processus** : Inclut la construction automatique du code, les tests, et la préparation de la livraison.
- **Avantages** :
 - **Réduction des Risques** : Les déploiements fréquents réduisent les risques associés à la livraison.
 - **Feedback Rapide** : Permet de recevoir rapidement des retours des utilisateurs finaux.
 - **Amélioration de la Qualité** : Les tests automatiques garantissent un haut niveau de qualité avant le déploiement.



DÉFINITION DU CONTINUOUS DEPLOYMENT

Vérifier dans
les questions

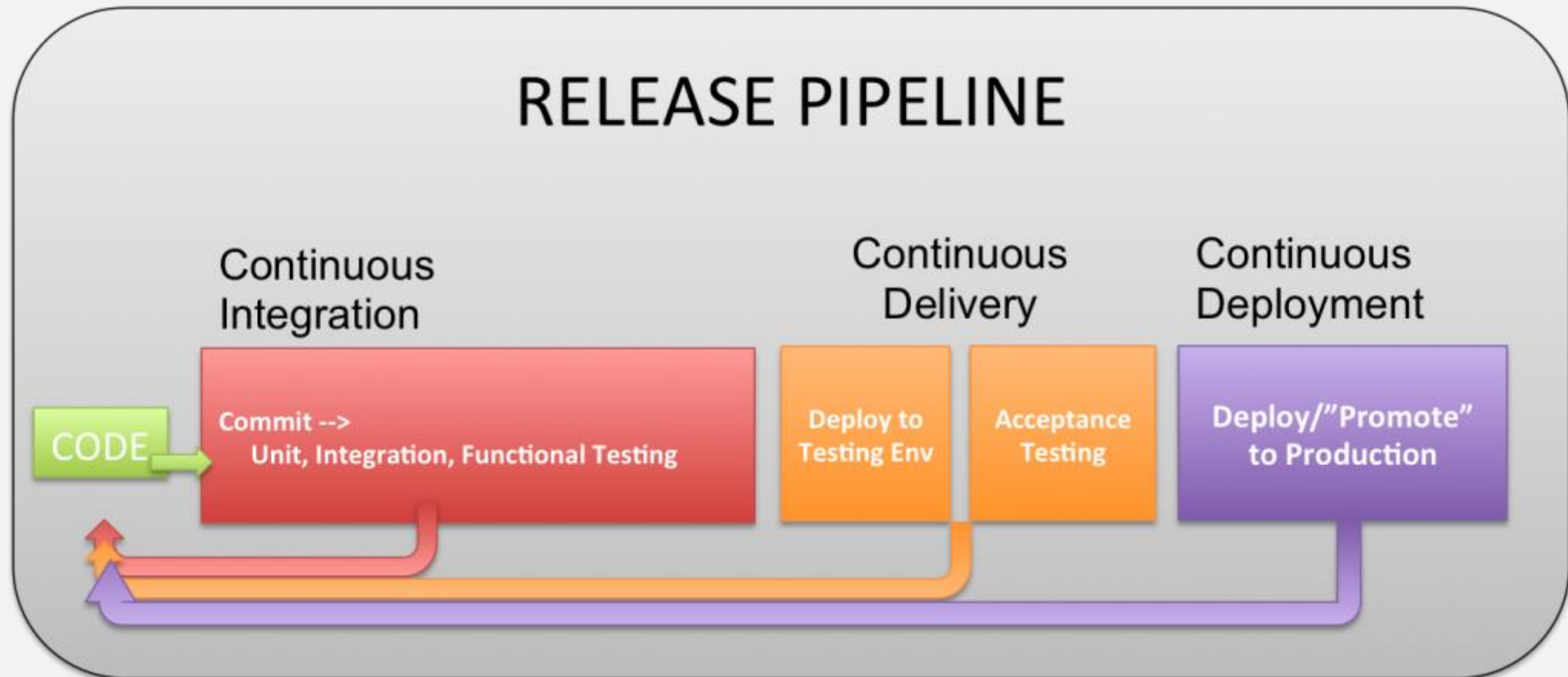
SOURCE CONTROL & CI/CD

- **Définition :** Le Continuous Deployment est une étape au-delà du Continuous Delivery, où chaque modification validée est automatiquement déployée en production.
- **Différence avec le Delivery :** Contrairement au Continuous Delivery, il n'y a pas d'étape manuelle de validation avant le déploiement.
- **Processus :** Automatisation complète du pipeline de déploiement, du code à la production.
- **Avantages :**
 - **Déploiements Fluides :** Les mises à jour continues rendent les déploiements plus fluides et moins notables pour les utilisateurs.
 - **Productivité Améliorée :** Libère les équipes de la nécessité de planifier des déploiements manuels.
 - **Réactivité Améliorée :** Permet une réponse rapide aux besoins du marché et des utilisateurs.



RELEASE PIPELINE

SOURCE CONTROL & CI/CD





PLEIN D'OUTILS !

SOURCE CONTROL & CI/CD

➤ Application Lifecycle Management

- Gère le cycle de vie
 - Compilation / Transpilation / Build
 - Lance les tests
 - Package l'application



➤ Frameworks de test

- Vu précédemment

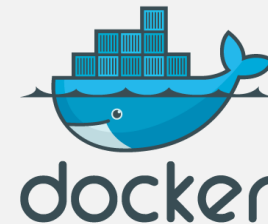
➤ Orchestrateurs et orchestrateurs « intégrés Cloud provider »

- Exécution des pipelines de l'ALM
- Récupération des rapports de test et de Qualimétrie



➤ Docker

- Suivi des versions
- Installation facile des outils
- environnements sur demande



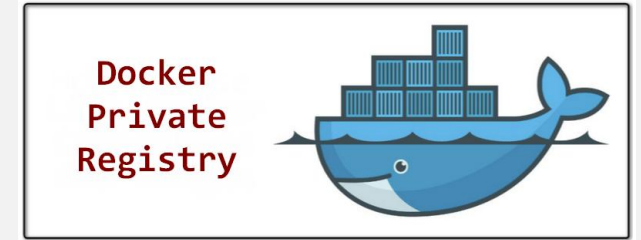


PLEIN D'OUTILS !

SOURCE CONTROL & CI/CD

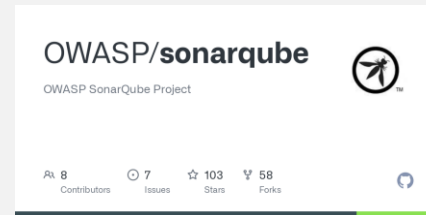
> Gestionnaire de dépôt

- importance de la donnée mais aussi des meta-data
- stockage du build mais aussi mirroring
- universel vs spécialisé



> Analyse

- SonarQube
- GitLab (avec plugin code climate ou autre)
- GitHub (idem)
 - utilisation de Copilot pour des suggestions d'écriture



> Sécurité et performance

TRAINING !
Q/R : ONESCRUM

<https://onescrum.lyon.sqli.com>

Utilisateur : **onescrum**
Mot de passe : **sqliPS1**

*Attention, utilisation uniquement
depuis le réseau SQLI ou le VPN.*



**THANK
YOU**

Thank you



SQLI DIGITAL EXPERIENCE